Performance Evaluation

Multimedia Retrieval · Chapter 2

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

2 - Performance Evaluation

A student searches a university library for books that provide the computer science foundations needed for a master's degree. One search system returns many useful books mixed with unrelated titles. Another returns fewer useful books, but places them near the top. Which system is better? The answer depends on what the student needs: broad coverage, a clean result set, or the best books in the first few positions.

Performance evaluation turns claims such as "better results" into measurements that can be checked and reproduced. A fair comparison requires more than a metric. It needs a representative collection, realistic information needs, consistent relevance judgments, and a clear definition of success. Without this shared benchmark, a higher score may reflect an easier test rather than a better retrieval method.

Evaluation also applies when a text system assigns labels rather than retrieving documents. A spam filter decides whether an email is unwanted, while an AI assistant may route a query to a knowledge base, web search, calculator, or clarification step. Retrieval evaluates decisions about query-document pairs; classification evaluates labels assigned to text items. Both expose the same fundamental trade-off: finding more desired items often means accepting more incorrect decisions.

What you'll learn

- Design a retrieval benchmark from a document collection, information needs, relevance judgments, and performance goals
- Select and interpret metrics for unordered results, ranked results, and rankings with graded relevance
- Calculate precision, recall, fallout, F-measures, reciprocal rank, average precision, mean average precision, and normalized discounted cumulative gain
- Analyze binary spam filtering and multiclass intent routing with confusion matrices, per-class metrics, and macro and micro averages
- Explain how prevalence, decision thresholds, ROC curves, and AUC affect the interpretation of classifier performance

Prerequisites

- The retrieval architectures and models introduced in 1 - Classical Text Retrieval
- Basic set operations, fractions, arithmetic means, and logarithms
- No prior knowledge of classification algorithms is required

Designing a Retrieval Benchmark

A university library holds 50 books. A student submits one information need: which books give a beginning master's student solid computer science foundations? Fifteen of the 50 books are useful for that need. Two retrieval systems answer the same request, and their first five results look like this.

Rank	System A result	Useful?	System B result	Useful?
1	Computer Organization and Design	yes	Database Systems: The Complete Book	yes
2	Operating System Concepts	yes	Pattern Recognition and Machine Learning	yes
3	The Handmaid's Tale	no	Introduction to Algorithms	yes
4	Narrative of the Life of Frederick Douglass	no	On the Origin of Species	no
5	Towards a New Architecture	no	Introduction to the Theory of Computation	yes

System A does not stop at rank 5. It returns 25 books in total, half the library, and finds 12 of the 15 useful ones. System B returns only 8 books, of which 6 are useful. So System A surfaces more of what exists, and System B wastes less of the reader's attention.

Which system is better? Two students with the same assignment can reasonably disagree. One wants a short list to read end to end and prefers System B. Another wants to be certain that nothing important was missed and prefers System A. Neither is wrong, so before we can measure anything, we have to say what counts as a good result.

What Counts as a Better Result

Relevance is not a property of a book. It is a judgment about whether a book serves one person's need at one moment. "Introduction to Algorithms" is essential to our student and useless to someone looking for a novel for a train journey. Every measurement in this chapter therefore rests on a human decision taken before any system ran.

That subjectivity is unavoidable, but it is containable. Once the need is written down and the judgments are recorded, the counting becomes mechanical: anyone who applies the same procedure to the same lists obtains the same numbers. Evaluation does not remove the subjective step, it isolates it.

What counts as success then varies enormously with the need. At one extreme only the first few results matter. Someone checking when BM25 was published reads two or three entries and stops, so anything below rank 5 might as well not exist. A short and precise list wins here, in the style of System B, and whatever relevant material it leaves behind costs the reader nothing. At the other extreme, missing anything is the real failure. A patent lawyer searching for prior art will work through a long and noisy list rather than risk overlooking one document, so broad coverage wins, in the style of System A, and a very short list would be useless however precise it was.

Many information needs are in between these extremes, including our student's. A foundations reading list has to be broad enough to cover the main areas of the field, yet short enough to work through in a semester. Neither system serves that well: System A buries the useful books among thirteen novels, and System B leaves out nine of the fifteen that would have helped. Web search leans towards the first extreme, biomedical literature search towards the middle, and the lesson is the same in every domain: "better" has no meaning until the intended use is named.

Why Anecdotes Are Not Evidence

Product advertising is full of comparative claims with no shared basis. A vendor states that a search engine is forty percent more accurate, without naming the collection it was tested on, the queries that were used,

who judged the results, or what "accurate" counts. Such a claim cannot be checked, and it is usually easy to construct: pick the queries where your system happens to win, and the number follows.

Retrieval systems invite exactly this failure, because single-example demonstrations are trivially available. For almost any pair of systems, someone can find one query where the first beats the second and another query where the reverse holds. The two result lists at the start of this section are a demonstration, not evidence. They describe one need in a 50-book collection, so they support no general statement about either system.

A scientific comparison replaces the anecdote with a fixed experimental setup. The collection is frozen, the information needs are written down in advance, the relevance judgments are made independently of the systems being compared, and the measurement procedure is stated before any results are seen. Others can then rerun the same experiment and obtain the same numbers. This is what a benchmark provides: not a guarantee that the winner is better for every purpose, but a claim that is reproducible and open to challenge.

Comparable setups also make progress cumulative. When many research groups report results on the same benchmark, a new method can be placed against a decade of prior work instead of against whatever the authors chose to reimplement.

From Cranfield to BEIR: how the field learned to compare systems (optional reading)

Shared evaluation is older than most of the methods it evaluates, and the paradigm has been revised roughly once per decade.

The Cranfield experiments, run by Cyril Cleverdon at the College of Aeronautics in Cranfield from the late 1950s into the 1960s, established the idea that retrieval quality can be measured offline. Cleverdon assembled a fixed set of documents, a fixed set of queries, and relevance judgments made by subject experts, then compared indexing methods against that fixed material. The abstraction was radical: rather than observing real users at work, the experiment freezes their needs into reusable test data. Every benchmark since has inherited both the power and the weakness of that move.

The Text REtrieval Conference, launched by the US National Institute of Standards and Technology in 1992, scaled the paradigm to collections far too large for exhaustive judging. TREC introduced pooling: participants submit their runs, the organizers merge the top-ranked documents from all submissions into a pool, and only the pool is judged. It also introduced tracks, so that ad-hoc search, question answering, legal discovery, and web search could each be evaluated with material suited to that task rather than by one universal test.

Comparable efforts followed for other language communities. CLEF began in 2000 in Europe, growing out of TREC's cross-language work, and NTCIR started in Japan in the late 1990s with a focus on Japanese and other Asian languages. Both extended the paradigm to multilingual and cross-lingual retrieval, where a query in one language must find documents in another.

MS MARCO, released by Microsoft in 2016, changed the economics again. Built from real Bing queries, it was large enough to train neural ranking models rather than merely to test them, which is why it became the standard proving ground for learned retrieval. The price was sparse judgments: with hundreds of thousands of queries, only a small number of documents per query could be labeled.

BEIR, introduced in 2021, responded to a new failure mode. Models tuned on MS MARCO performed impressively on MS MARCO and often disappointed elsewhere. BEIR bundles eighteen heterogeneous collections and evaluates models without task-specific training, measuring whether a method generalizes rather than whether it has learned one benchmark well.

The Four Components of a Benchmark

A benchmark is built from four parts: a document collection, a set of information needs, relevance judgments, and a measurement procedure. Each part answers one question, and a weakness in any of them undermines everything measured on top.

The **document collection** defines the universe the system may search. It can hold news articles, scientific papers, legal filings, product descriptions, or, as in our running example, library records. Two properties matter most. The collection must be stable, because results are only comparable over time if the searched material does not change underneath them. It must also be representative, reflecting the document lengths, content types, and topic mix that the target users really encounter. Well-known examples include the TREC collections, which range from newspaper archives to biomedical abstracts, MS MARCO for web-style passage retrieval, and domain corpora such as PubMed Central and arXiv for scientific literature.

The **information needs** state what the users actually want. It is worth separating the need from the query string: the need is "which books give a beginning master's student solid computer science foundations", while the query is whatever short text the user types into the search box. Assessors judge against the need, not the keywords, which is why the need must be written out clearly enough that two people would interpret it the same way. Needs are best derived from real user behaviour such as query logs. When no log exists, large language models can draft candidate needs from samples of the collection, which expands coverage cheaply, though the drafts still require review to stay realistic.

The **relevance judgments** record, for each need, which documents satisfy it. They are the ground truth, so their quality bounds the quality of every conclusion. Human assessors work from written guidelines, receive training on example cases, and are checked for agreement, because vague criteria produce inconsistent labels that no metric can repair. Large language models can pre-screen documents and propose judgments for clear-cut cases, leaving human effort for the borderline ones. Judgments must then be frozen: revising them later silently invalidates every result measured earlier.

The **measurement procedure** turns a result list into a number. It fixes which measure is computed, at which cutoff, and how values are combined across needs. The next three sections develop these measures, starting with results treated as an unordered set, then as a ranking, then as a ranking with graded relevance. What matters here is that the procedure is chosen before the experiment, not after the results are known.

The following table collects the practical rules for each component.

Component	What it provides	Design rule	Common failure
Document collection	The universe that may be searched	Freeze it for the benchmark's lifetime, make it representative of the target domain, preprocess consistently, and give every document a stable identifier	Built around one feature, so it flatters the methods that exploit that feature
Information needs	What the target users want	Derive needs from real user behaviour, state each one unambiguously, and provide 25 to 50 for a first experiment or 100 and more for a serious comparison	Wording so vague that assessors disagree and the ground truth becomes noisy
Relevance judgments	The ground truth for scoring results	Write assessor guidelines, train assessors, check their agreement, and freeze the labels once published	Judgments revised mid-project, which invalidates all earlier measurements
Measurement procedure	The rule that turns results into numbers	State the measure, the cutoff, and the averaging method before running any experiment	Measure chosen after seeing the results, which turns evaluation into advocacy

Building all four parts is expensive, which is why established benchmarks are reused heavily and why a small, carefully judged collection is often more valuable than a large, carelessly labeled one.

Judging at Scale

One practical obstacle remains. Judging every document for every need is impossible at realistic scale: fifty books can be assessed exhaustively, fifty million cannot. Pooling is the standard response. The organizers collect the documents that participating systems actually retrieved, judge that pool, and treat everything unjudged as not relevant. Figure 2.1 shows the situation this assumes. Relevant documents that no participant retrieved never get judged, so the measured values differ somewhat from what exhaustive judging would produce. For a comparison that matters less than it first appears: every participant is scored against the same judged subset, so the ordering between them survives even though the individual numbers are approximations.

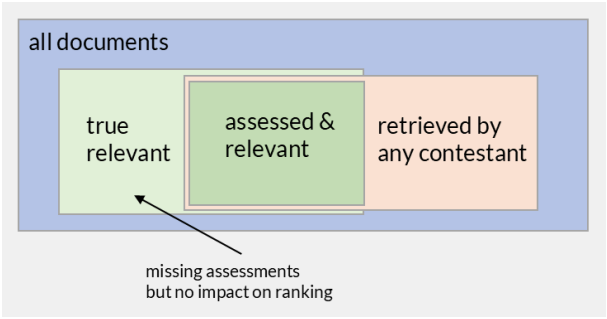


Figure 2.1: Pooled assessment when participating systems retrieve most relevant documents.

This assumption is sound while the pool covers most of the relevant material, and it weakens as the judged fraction shrinks. Pooling is one of several places where a benchmark can mislead rather than inform, and When Evaluation Misleads returns to all of them once we have the measures needed to describe what gets distorted.

> **Note**
>
> Sections 5 and 6 evaluate text classifiers rather than retrieval results, and the ground truth there is a test set rather than a benchmark of the kind described above. Three differences matter. The unit of evaluation is a single text item and its label, not a need-document pair. The labels are normally complete, so pooling does not apply, but class balance and label quality become the central concerns. And a classification dataset is split into training, validation, and test parts, with any decision threshold tuned on the validation part and never on the test part. Section 5 develops these points where they are needed.

With a benchmark in place, the remaining question is mechanical: how do we turn a result list into a number that reflects how well the need was served? The next section answers it for results treated as an unordered set, using the same library collection and the same two systems.

Precision and Recall

Boolean retrieval returns a set of documents with no order attached: a document either matches the query or it does not. This section evaluates that kind of result. Ranking comes in Evaluating Ranked Results; until then, a result list is treated purely as a set.

Recall System A and System B from Designing a Retrieval Benchmark, both answering the same need: which books give a beginning master's student solid computer science foundations? System A returned 25 books, System B returned 8. To turn "returned" and "useful" into numbers, every book in the 50-book collection falls into exactly one of four groups, depending on whether the system retrieved it and whether it is actually relevant.

Counting Outcomes

	Relevant	Not relevant
Retrieved	True Positive (TP)	False Positive (FP)
Not retrieved	False Negative (FN)	True Negative (TN)

A true positive is a relevant book the system retrieved: a hit. A false positive is a book the system retrieved that turns out not to be relevant: noise in the result. A false negative is a relevant book the system failed to retrieve: a miss. A true negative is a book the system correctly left out. Figure 2.2 shows this partition as overlapping sets for both systems: the 50-book collection, the 15 relevant books, the retrieved books, and their intersection.

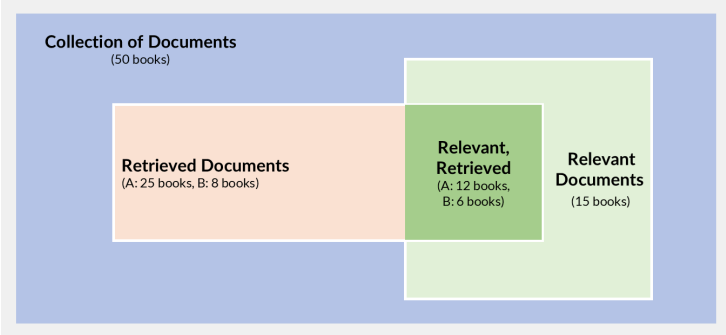


Figure 2.2: Set-theoretic view of retrieval performance for Systems A and B over the 50-book collection.

Counting each book in the collection against the two systems gives:

	TP	FP	FN	TN
System A	12	13	3	22
System B	6	2	9	33

Every row sums to 50, the size of the collection, and every count came directly from checking the 15 known relevant books against each system's result list.

Precision and Recall

The raw counts alone are hard to compare. System A has 12 true positives and System B has 6, but A also retrieved three times as many books and operates in a collection where 15 are relevant. A system retrieving the entire library would produce 15 true positives and look best by that count alone, which shows that an absolute count cannot separate a good system from one that simply returns everything. Ratios solve this: dividing by the appropriate total produces a value between 0 and 1 that is independent of how many books the system returned or how many relevant books the collection contains.

Key Formula: Precision and Recall

$$p = \frac{TP}{TP + FP} \qquad r = \frac{TP}{TP + FN} \tag{2.1}$$

Precision is the fraction of retrieved documents that are relevant. Recall is the fraction of relevant documents that were retrieved.

Example: Precision and Recall for System A and B

System A retrieved 25 books and found 12 of the 15 relevant ones:

$$p_A = \frac{12}{12 + 13} = \frac{12}{25} = 48\% \qquad r_A = \frac{12}{12 + 3} = \frac{12}{15} = 80\% \tag{2.2}$$

System B retrieved 8 books and found 6 of the 15 relevant ones:

$$p_B = \frac{6}{6 + 2} = \frac{6}{8} = 75\% \qquad r_B = \frac{6}{6 + 9} = \frac{6}{15} = 40\% \tag{2.3}$$

This is the trade-off from Designing a Retrieval Benchmark stated in numbers rather than in words. System B's list is three-quarters useful material; System A's list finds four-fifths of everything useful but buries it among thirteen novels. Whether 48% precision at 80% recall is better than 75% precision at 40% recall still depends on what the user prefers (quick fact check or exhaustive library scan).

A third quantity, fallout, measures the opposite kind of mistake: how much of the irrelevant material got pulled in.

$$f = \frac{FP}{FP + TN} \tag{2.4}$$

Fallout is the fraction of non-relevant documents that the system retrieved. For our two systems, $f_A = 13/35 = 37.1\%$ and $f_B = 2/35 = 5.7\%$. Fallout measures the reading cost imposed on someone who inspects every result: the patent lawyer from Section 1 needs high recall, because missing a document is expensive, but also wants low fallout, because every irrelevant document in the list costs time and attention. System A delivers recall of 80% but pulls in more than a third of the irrelevant collection; the ideal for an exhaustive search would push recall toward 1 while keeping fallout near 0.

Precision and recall are in tension

Retrieving more documents raises recall, since additional relevant ones may be included, and a system that returns the entire collection reaches perfect recall by construction. The cost is that more non-relevant documents come in as well, which lowers precision. In the other direction, a smaller result set tends to have higher precision, but at the risk of missing many of the relevant documents. In practice, improving one measure typically comes at the expense of the other, which is why both are always reported together.

Balancing the Two

Comparing two numbers side by side, as above, does not say which system wins. The F_β-measure collapses precision and recall into a single value, letting a scenario's priority set the trade-off explicitly.

Key Formula: F-beta Measure

$$F_\beta = \frac{(\beta^2 + 1) \cdot p \cdot r}{\beta^2 \cdot p + r} \tag{2.5}$$

The weighted harmonic mean of precision and recall. $\beta < 1$ favours precision, $\beta > 1$ favours recall, and $\beta = 1$ weighs them equally.

At $\beta = 0$ the formula reduces to precision alone; as $\beta \to \infty$ it reduces to recall alone. F_1, the case $\beta = 1$, is the most common default and appears throughout machine learning wherever a single number is needed to compare classifiers.

Example: F-beta for System A and B

Using $p_A = 0.48$, $r_A = 0.80$ and $p_B = 0.75$, $r_B = 0.40$ from above:

β	Weighting	$F_\beta(A)$	$F_\beta(B)$	Winner
0.5	favours precision	0.522	0.638	B
1.0	equal weight	0.600	0.522	A
2.0	favours recall	0.706	0.441	A

The winner changes between $\beta = 0.5$ and $\beta = 1$. Neither system is better in general: a fact-checker who wants a short, clean list would set β below 1 and prefer System B, while a patent lawyer would set β above 1 and prefer System A. F_1 is a convenient default, not a neutral one, since it silently commits to weighing a false positive and a false negative equally.

Averaging Across Needs

One need produces one precision and one recall value. A real benchmark evaluates many needs, and summarising all of them into a single score requires an averaging method. The natural first idea is to compute precision and recall for each need separately and then take the arithmetic mean.

Example: Consider two additional needs alongside the CS-foundations query, answered by the same two systems:

Need	Relevant	System A: retrieved / found	System B: retrieved / found
CS foundations	15	25 / 12	8 / 6
Stage plays	5	6 / 4	4 / 3
Russian novels	1	3 / 0	1 / 1

The straightforward summary is the arithmetic mean of the per-need values:

Key Formula: Macro Averaging

$$p_{\text{macro}} = \frac{1}{N}\sum_{i=1}^{N} p_i \qquad r_{\text{macro}} = \frac{1}{N}\sum_{i=1}^{N} r_i \tag{2.6}$$

Every need counts equally, regardless of how many relevant documents it has.

Example: Macro Averaging

Per-need recall for both systems:

Need	r_A	r_B
CS foundations	12/15 = 0.800	6/15 = 0.400
Stage plays	4/5 = 0.800	3/5 = 0.600
Russian novels	0/1 = 0.000	1/1 = 1.000

$$r_{\text{macro},A} = \frac{0.800 + 0.800 + 0.000}{3} = 53.3\% \tag{2.7}$$

$$r_{\text{macro},B} = \frac{0.400 + 0.600 + 1.000}{3} = 66.7\% \tag{2.8}$$

System B wins. Yet looking at the table, System A finds more relevant documents on two of the three needs and retrieves 16 of 21 relevant books in total, against B's 10. The single Russian-novels miss, a zero out of one, drags A's average down by 27 percentage points.

The problem is visible: macro-averaging gives each need the same weight $1/N$ in the sum, so a need with one relevant document contributes as much to the average as a need with fifteen. A single zero from the Russian-novels miss lowers the three-need average by a full $1/3$, which is a large negative impact relative to the system's overall performance across 21 relevant documents. This is intentional when every need matters equally, but it can misrepresent a system's aggregate capability.

Micro-averaging addresses this by pooling all outcome counts before computing the ratio. Instead of averaging per-need ratios, it sums the numerators and denominators across needs:

$$p_{\text{micro}} = \frac{\sum_{i=1}^{N} TP_i}{\sum_{i=1}^{N} (TP_i + FP_i)} \qquad r_{\text{micro}} = \frac{\sum_{i=1}^{N} TP_i}{\sum_{i=1}^{N} (TP_i + FN_i)} \tag{2.9}$$

Every document counts equally. Needs with more relevant documents contribute more to the result.

Example: Micro Averaging

Pooling the recall counts across all three needs:

	$\sum TP$	$\sum (TP + FN)$	r_{micro}
System A	12 + 4 + 0 = 16	15 + 5 + 1 = 21	16/21 = 76.2%
System B	6 + 3 + 1 = 10	15 + 5 + 1 = 21	10/21 = 47.6%

System A now wins on recall, because the CS-foundations need, where A finds 12 of 15 books, contributes 15 of the 21 relevant documents to the pool and dominates the sum. The single Russian-novels miss adds only one false negative to a denominator of 21 and barely registers.

Same systems, same needs, opposite conclusions on recall: macro says B is better, micro says A is better. The difference reflects a genuine choice about what matters.

Macro averaging is the standard in retrieval evaluation

In retrieval evaluation, the standard practice is macro-averaging: MAP, mean nDCG, and mean MRR all compute a per-topic score and then take the arithmetic mean. Micro-averaging across topics is rarely used, because weighting each document equally would let one large topic dominate all others. The macro/micro choice becomes more relevant in classification evaluation, where classes are often imbalanced and the two methods can disagree sharply; Section 5 returns to this point for the intent-routing example. When reporting retrieval results, specify both the measure and the number of topics it was averaged over.

Architecture and the Metrics

Retrieval Systems and Flows introduced retriever-only, retriever-with-filter, and retriever-ranker architectures. Precision and recall explain why each stage exists and how each is evaluated. The retriever is optimized for recall: it casts a wide net over the collection, accepting low precision because a false positive can still be removed later, while a false negative is lost for good. The ranker or filter is optimized for precision: it reorders or removes candidates so that the top of the list is dense with relevant material. Even a user who cares only about a precise top-ten result depends on the retriever's recall, because the ranker can only surface documents that the retriever found in the first place.

Hands-on: Precision and Recall

> Recompute every number in this section from the library collection, then edit a run and watch precision, recall, and F_β respond. Open notebook ->
>
> *Includes pre-run results: you can read through or download and experiment.*

Precision, recall, and their averages describe a result set with no notion of order. The next section brings rank back into the picture: whether a system found the right documents matters less if they never appear near the top of the list.

Evaluating Ranked Results

Precision and recall treat a result set as a bag: a document is either in the set or not, and the metrics do not change if the order is shuffled. A ranked retrieval system does more than decide which documents to return; it decides which ones to show first. A user who reads from the top and stops after five items gets a very different experience depending on whether the relevant books are at ranks 1 and 2 or at ranks 16 and 18. Evaluation must account for position, not only membership.

Recall the first five positions from Designing a Retrieval Benchmark:

Rank	System A result	Useful?	System B result	Useful?
1	Computer Organization and Design	yes	Database Systems: The Complete Book	yes
2	Operating System Concepts	yes	Pattern Recognition and Machine Learning	yes
3	The Handmaid's Tale	no	Introduction to Algorithms	yes
4	Narrative of the Life of Frederick Douglass	no	On the Origin of Species	no
5	Towards a New Architecture	no	Introduction to the Theory of Computation	yes

System A retrieves 25 books total (12 of 15 relevant); System B retrieves 8 (6 of 15 relevant). The set-based precision numbers from Precision and Recall already showed that B is more precise overall, but ranking adds a second dimension: how quickly does the user reach useful material? System A places two relevant books at the top, then three misses; System B delivers four of five.

Precision at Rank k

The simplest rank-aware metric evaluates only the first k positions. Precision at rank k counts how many of the top-k documents are relevant, ignoring everything below.

Key Formula: Precision at k

$$P@k = \frac{|\{D_1, \ldots, D_k\} \cap \text{Rel}|}{k} \tag{2.10}$$

The fraction of relevant documents among the first k results.

$P@k$ answers one practical question: if the user reads only the first k items, what proportion is useful? It does not require knowing how many relevant documents exist in the collection, which makes it easy to compute and interpret. It also does not penalize a system for missing relevant documents below rank k: a system that places one relevant book at rank 1 and nothing else achieves $P@1 = 1.0$ regardless of how many relevant books it missed.

Example: P@k for System A and B

k	$P@k$ (A)	$P@k$ (B)
3	2/3 = 66.7%	3/3 = 100%
5	2/5 = 40.0%	4/5 = 80.0%
10	5/10 = 50.0%	6/10 = 60.0%*

*System B returns only 8 documents. Positions 9 and 10 are empty, counted as non-relevant.

System B dominates at every cutoff. A student who reads the first five results gets four useful books from B but only two from A. $P@k$ captures this user experience directly.

P@k ignores recall

$P@k$ rewards a system for being precise at the top but says nothing about how many relevant documents exist below rank k. Two systems with identical $P@5$ can differ drastically in total recall: one may have surfaced all relevant documents, the other may have missed most of them. Report $P@k$ alongside recall or use metrics that integrate both.

Reciprocal Rank and MRR

Some information needs have a single correct answer, or the user is satisfied as soon as one relevant result appears. A library patron asking "Who wrote Dracula?" needs exactly one fact; seeing it at rank 1 is ideal, at rank 20 is frustrating. The reciprocal rank captures this notion.

Key Formula: Reciprocal Rank and MRR

$$RR = \frac{1}{\text{rank of first relevant document}} \qquad MRR(\mathbb{Q}) = \frac{1}{|\,\mathbb{Q}\,|}\sum_{i=1}^{|\,\mathbb{Q}\,|} RR_i \tag{2.11}$$

The reciprocal of the position where the user first finds what they need, averaged across a set of queries $\mathbb{Q}$.

$RR = 1$ when the first result is relevant; $RR = 0.5$ when the first relevant result is at rank 2; $RR \to 0$ as the relevant document is pushed further down. If no relevant document appears in the result list at all, $RR = 0$.

Example: MRR for fact lookups

MRR is best demonstrated on fact-lookup queries where exactly one answer exists. Suppose four students each ask the library catalogue a different question, and both systems return a ranked list. The only thing that matters is how far down the student must scroll to find the answer:

Query	Answer	System A: rank	RR_A	System B: rank	RR_B
"Who wrote Dracula?"	Dracula	1	1.000	3	0.333
"A book on operating system design?"	Operating System Concepts	4	0.250	1	1.000
"A nineteenth-century Russian novel?"	Crime and Punishment	6	0.167	2	0.500
"A dystopian novel by Atwood?"	The Handmaid's Tale	2	0.500	5	0.200

$$MRR_A = \frac{1.000 + 0.250 + 0.167 + 0.500}{4} = 0.479 \tag{2.12}$$

$$MRR_B = \frac{0.333 + 1.000 + 0.500 + 0.200}{4} = 0.508 \tag{2.13}$$

Neither system dominates: A answers one query instantly ($RR = 1$) but buries the Russian novel at rank 6, while B also has one perfect answer but pushes the Atwood question to rank 5. The averages are close, which tells us the two systems surface first answers at comparable speed overall despite different per-query strengths.

MRR is widely used in question answering and known-item search, where one good answer suffices. It ignores what happens after the first relevant document: a system that places five relevant books at ranks 1 through 5 scores the same $RR = 1$ as one that places a single relevant book at rank 1 and nothing else. For needs where multiple relevant documents matter, as in our CS-foundations example with 15 relevant books, MRR is the wrong tool.

The Precision-Recall Curve

$P@k$ evaluates one fixed cutoff. A more complete picture plots precision against recall at every rank where a relevant document appears. As the system returns more documents, recall increases (more relevant documents are found) and precision typically decreases (more non-relevant documents accumulate). Plotting precision against recall produces the precision-recall curve.

At each rank i in the result list, define:

$$P_i = \frac{|\{d_1, \ldots, d_i\} \cap \text{Rel}|}{i} \qquad R_i = \frac{|\{d_1, \ldots, d_i\} \cap \text{Rel}|}{|\text{Rel}|} \tag{2.14}$$

A point (R_i, P_i) is recorded only at ranks where a relevant document is retrieved: those are the only ranks where recall changes. Between two relevant documents, recall stays constant while precision drops (the denominator i grows with no new relevant hit), so these in-between ranks do not add new curve points.

Example: Precision-Recall curve for System A

The relevant documents in System A's ranking appear at ranks 1, 2, 6, 7, 9, 12, 13, 16, 18, 21, 23, and 25. At each of those positions:

Rank	Relevant found	P_i	R_i
1	1	1/1 = 1.000	1/15 = 0.067
2	2	2/2 = 1.000	2/15 = 0.133
6	3	3/6 = 0.500	3/15 = 0.200
7	4	4/7 = 0.571	4/15 = 0.267
9	5	5/9 = 0.556	5/15 = 0.333
12	6	6/12 = 0.500	6/15 = 0.400
13	7	7/13 = 0.538	7/15 = 0.467
16	8	8/16 = 0.500	8/15 = 0.533
18	9	9/18 = 0.500	9/15 = 0.600
21	10	10/21 = 0.476	10/15 = 0.667
23	11	11/23 = 0.478	11/15 = 0.733
25	12	12/25 = 0.480	12/15 = 0.800

The curve starts at $(0.067, 1.0)$ and ends at $(0.800, 0.480)$. After rank 2, three consecutive non-relevant books push precision down before the next relevant book at rank 6 lifts it back to 0.5. The curve never reaches recall 1.0 because System A misses 3 of the 15 relevant books.

Reading the Curve

Figure 2.3 annotates the key regions of a precision-recall curve. The upper-left corner (high precision, low recall) is where a fact-checker operates: a short, clean result list with few mistakes. The lower-right corner (high recall, low precision) is where a patent lawyer operates: an exhaustive search that tolerates noise to avoid missing anything. The diagonal arrow labelled "system efficiency" points toward the ideal corner $(1, 1)$: the closer a curve pushes toward that point, the better the system balances both goals simultaneously.

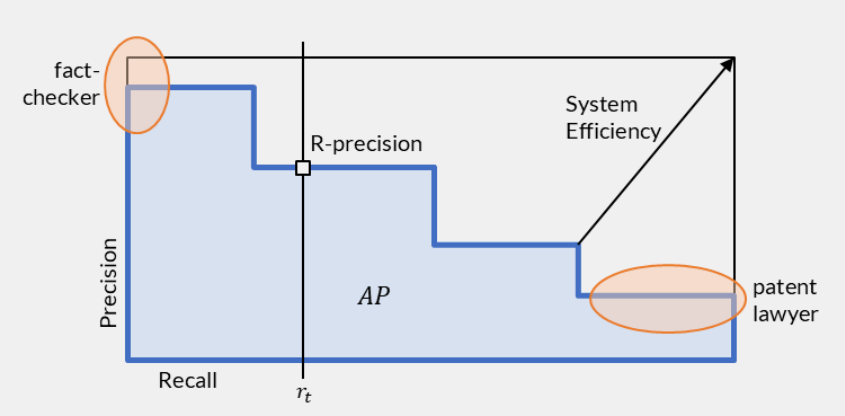


Figure 2.3: Precision-recall curve with key evaluation landmarks: the shaded area is Average Precision (AP), R-precision marks the point where the number of results equals the number of relevant documents, and the two user profiles illustrate the trade-off between precision-oriented and recall-oriented search.

The shaded area under the curve represents Average Precision (AP), which the next subsection defines formally. R-precision, marked on the curve at the recall level $r_t = R/\ |\ \text{Rel}|$ where R is the number of relevant documents, provides a single-point summary.

Interpolation for Cross-System Comparison

The 12 points in the table above form a smooth, largely decreasing scatter for one system and one need. The need for interpolation arises when we want to compare two systems or average curves across multiple needs: each system produces points at different recall levels, so we cannot directly subtract or average their precisions. Interpolation defines precision at any recall level r as the maximum precision achieved at any recall level $r' \geq r$:

$$P_{\text{interp}}(r) = \max_{r' \geq r} P(r') \tag{2.15}$$

This produces a non-increasing step function evaluated at standard recall levels (0, 0.1, 0.2, ..., 1.0). Two systems evaluated on the same 11 recall points can be compared point by point or averaged across queries. For System A, interpolated precision at recall 0 is 1.0 (the best precision it ever achieves), while at recall 0.9 and 1.0 it is 0 (the system never reaches those recall levels).

R-Precision

R-precision links precision to recall through a single number. If the collection contains R relevant documents for a need, R-precision is the precision after exactly R documents have been retrieved.

$$\text{R -Prec} = P@R = \frac{|\ \{D_1, ..., D_R\} \cap \text{Rel}|}{R} \tag{2.16}$$

A perfect system would place all R relevant documents in the first R positions, achieving R-precision of 1.0. The measure adjusts itself to the difficulty of the need: a need with 15 relevant documents (our CS-foundations example) evaluates at cutoff 15, while a need with 2 relevant documents evaluates at cutoff 2.

Example: R-Precision

For the CS-foundations need, $R = 15$. System A places 7 relevant books in its first 15 positions:

$$\text{R -Prec}(A) = \frac{7}{15} = 0.467 \tag{2.17}$$

System B returns only 8 documents total. Positions 9 through 15 contain nothing (treated as non-relevant), so the 6 relevant books it returned are all that count:

$$\text{R -Prec}(B) = \frac{6}{15} = 0.400 \tag{2.18}$$

System A wins here because its longer result list gives it more chances to place relevant books in the first 15 positions (7 out of 25 retrieved), while System B returns only 8 books total, so positions 9 through 15 are empty and count against it.

R-precision is simple and self-adjusting, but it reduces an entire ranking to one point. The metrics that follow evaluate the full curve.

Average Precision

Average precision (AP) collapses the precision-recall curve into a single number. Geometrically, it approximates the area under the precision-recall curve: the shaded region in Figure 2.3. A system whose curve stays high across all recall levels has a large area and a high AP; one whose precision collapses early has a small area and a low AP.

The computation is straightforward: at each rank where a relevant document appears, record the precision at that rank. Then take the mean of those values, using the total number of relevant documents in the collection as the divisor.

Key Formula: Average Precision

$$AP = \frac{1}{|\text{ Rel}|} \sum_{k\,:\,d_k \in \text{ Rel}} P_k \tag{2.19}$$

where P_k is the precision at rank k, and the sum runs only over the ranks at which a relevant document appears. The divisor is the total number of relevant documents in the collection, not the number retrieved.

Dividing by $|\text{ Rel}|$ rather than by the number of relevant documents actually found means that missed documents contribute implicit zeros: they never appear in the ranking, so they add nothing to the numerator, but the denominator still counts them. This is what makes AP sensitive to recall, not only to precision at the top.

AP uses the raw (non-interpolated) precision values. Interpolation serves a different purpose: it defines precision at arbitrary recall levels so that curves from different systems can be compared point by point or averaged. AP does not require that step because it sums only at the fixed rank positions where relevant documents actually occur.

Example: Average Precision for System A and B

System A finds 12 of 15 relevant documents. The precision at each relevant rank:

$$AP_A = \frac{1}{15}\left(\frac{1}{1} + \frac{2}{2} + \frac{3}{6} + \frac{4}{7} + \frac{5}{9} + \frac{6}{12} + \frac{7}{13} + \frac{8}{16} + \frac{9}{18} + \frac{10}{21} + \frac{11}{23} + \frac{12}{25}\right) = 0.473 \tag{2.20}$$

The three relevant books that System A missed (ranks would be needed beyond 25, or they were never retrieved) each contribute 0 to the sum, pulling the average down.

System B finds 6 of 15 relevant documents, all placed high:

$$AP_B = \frac{1}{15}\left(\frac{1}{1} + \frac{2}{2} + \frac{3}{3} + \frac{4}{5} + \frac{5}{6} + \frac{6}{8}\right) = 0.359 \tag{2.21}$$

System B's precisions at the positions where it finds relevant documents are higher (it averages 0.897 over its 6 hits vs. A's 0.592 over its 12 hits), but it misses 9 of 15 relevant books. Those 9 zeros in the denominator dominate: $AP_B < AP_A$.

AP rewards both placing relevant documents high (each precision term is larger when fewer non-relevant documents precede it) and finding more relevant documents (more terms contribute to the sum). This makes

it the single best summary of a ranking for a given need: unlike $P@k$ it accounts for the full list, and unlike MRR it values every relevant document, not only the first.

Mean Average Precision

Evaluating a system on one need gives one AP value. A benchmark with multiple needs produces an AP per need, and mean average precision (MAP) averages them.

$$MAP = \frac{1}{|\mathbb{Q}|} \sum_{i=1}^{|\mathbb{Q}|} AP_i \tag{2.22}$$

MAP is a macro-average: each need contributes equally regardless of how many relevant documents it contains. This is the same averaging strategy discussed in Precision and Recall, and the same caveats apply: a need with one relevant document has as much influence on MAP as a need with fifteen.

Example: MAP across four needs

Using the four needs from our library example:

Need	Relevant	AP_A	AP_B
CS foundations	15	0.473	0.359
Stage plays	5	0.587	0.600
Atwood books	2	0.750	1.000
Russian novels	1	0.000	1.000

$$MAP_A = \frac{0.473 + 0.587 + 0.750 + 0.000}{4} = 0.452 \tag{2.23}$$

$$MAP_B = \frac{0.359 + 0.600 + 1.000 + 1.000}{4} = 0.740 \tag{2.24}$$

System B wins on MAP despite finding far fewer relevant documents on the largest need. Two factors combine: B has perfect AP on two smaller needs, while A's zero on Russian novels costs 0.25 in the average, and A also fails to find one of the five plays in its stage-plays run.

MAP is the standard primary metric in TREC and most retrieval benchmarks. A single MAP value summarizes an entire system's behaviour across all evaluated needs. When two systems are compared, a paired test (such as a paired t-test or Wilcoxon signed-rank test) on the per-need AP values determines whether the difference is statistically significant, not just the result of a few easy or hard queries.

Choosing a Metric

Each metric answers a different question about the ranking:

Metric	Question it answers	Sensitive to
$P@k$	How clean are the first k results?	Precision at a fixed cutoff
MRR	How quickly does the user find one good result?	Position of the first relevant document
R -Prec	How well does the system fill the first R slots?	Balance of precision and recall at a natural cutoff
AP	How good is the overall ranking for one need?	Both precision and recall, position-weighted
MAP	How good is the system across many needs?	Per-need AP, macro-averaged

$P@k$ and MRR suit settings where users inspect only the top few results: web search, question answering, voice assistants. AP and MAP suit settings where the full ranking matters: patent search, systematic

reviews, benchmark comparisons. R-precision provides a single summary that adjusts to the number of relevant documents per need.

All of these metrics treat relevance as binary: a document is relevant or it is not. The next section relaxes this assumption, introducing graded relevance where some documents are more useful than others, and metrics that reward a system for placing highly relevant documents above marginally relevant ones.

Hands-on: Ranked Evaluation

Compute P@k, AP, and MAP from the library rankings, then reorder documents and watch the metrics respond. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Evaluating Graded Relevance

All metrics so far treat relevance as binary: a document is relevant or it is not. Average precision, for instance, gives System A's rank-1 result ("Computer Organization and Design") the same credit as System B's rank-1 result ("Database Systems: The Complete Book"), because both are relevant. Yet for a student seeking CS foundations, the two books are not equally useful. The grading from Designing a Retrieval Benchmark distinguishes core foundations (grade 3), important specializations (grade 2), and useful peripheral reading (grade 1). "Database Systems" is a grade-3 core text; "Computer Organization and Design" is grade-1 peripheral reading. A metric that sees only "relevant" cannot tell these apart.

Of the 15 relevant books in the collection, 3 are graded as core foundations, 4 as important specializations, and 8 as useful peripheral reading. The complete grade distribution:

Grade	Meaning	Count	Examples
3	Core foundations	3	Database Systems, Pattern Recognition, Data Science from Scratch
2	Important specialization	4	Operating Systems, AI: A Modern Approach, Programming Languages, Theory of Computation
1	Useful peripheral reading	8	Algorithms, Computer Networks, SICP, Cryptography, …
0	Not relevant	35	Fiction, poetry, drama, general non-fiction

Consider the first five results from each system, now labelled with grades:

Rank	System A	Grade	System B	Grade
1	Computer Organization and Design	1	Database Systems: The Complete Book	3
2	Operating System Concepts	2	Pattern Recognition and Machine Learning	3
3	The Handmaid's Tale	0	Introduction to Algorithms	1
4	Narrative of the Life of Frederick Douglass	0	On the Origin of Species	0
5	Towards a New Architecture	0	Introduction to the Theory of Computation	2

Binary AP cannot distinguish these two top-5 lists beyond counting "2 relevant at top for A, 4 for B". Graded evaluation does more: it rewards B for placing its two highest-value books (both grade 3) in the most visible positions, while A placed a peripheral text (grade 1) at rank 1 and pushed its grade-3 titles to ranks 6, 12, and 18.

A second limitation of AP is subtler. AP rewards a system for placing any relevant document higher, but it does not penalize a system that places relevant documents in a sub-optimal order among themselves. If two relevant documents appear at ranks 3 and 5, AP does not care which of the two is more important. A graded metric can: it assigns more credit when a highly relevant document sits above a marginally relevant one.

Cumulative Gain

The simplest graded metric just sums the relevance grades of the documents returned up to rank k:

Key Formula: Cumulative Gain

$$CG_k = \sum_{i=1}^{k} rel_i \tag{2.25}$$

The total relevance value accumulated in the first k positions. Higher grades contribute more.

CG_k measures how much total value the user has gathered after reading k results. It is easy to compute but ignores order entirely: swapping a grade-3 book from rank 10 to rank 1 does not change CG_{10}.

Example: Cumulative Gain at k = 5

From the table above:

$$CG_5(A) = 1 + 2 + 0 + 0 + 0 = 3 \tag{2.26}$$

$$CG_5(B) = 3 + 3 + 1 + 0 + 2 = 9 \tag{2.27}$$

System B delivers three times the accumulated value in the first five positions. But if we shuffled B's list to put grade-0 at rank 1 and grade-3 at rank 5, CG_5 would remain 9. Rank does not matter to CG.

Discounted Cumulative Gain

To make position matter, we discount contributions from lower ranks. The idea is that a user reading from the top gets diminishing benefit from each additional position: finding a grade-3 book at rank 1 is more valuable than finding the same book at rank 10, because by rank 10 the user may have already stopped reading, or the earlier results have already partially satisfied the need.

Key Formula: Discounted Cumulative Gain

$$DCG_k = \sum_{i=1}^{k} \frac{rel_i}{\log_2(i+1)} \tag{2.28}$$

Each relevance grade is divided by a logarithmic discount factor that grows with rank. Rank 1 has no discount ($\log_2 2 = 1$); rank 10 discounts by $\log_2 11 \approx 3.46$.

The logarithmic discount encodes a model of user patience: the benefit of each additional rank position decreases, but never reaches zero. A document at rank 10 still contributes, just less than the same document at rank 1.

Example: DCG at k = 5

Rank	Discount $\frac{1}{\log_2(i+1)}$	A: grade	A: contribution	B: grade	B: contribution
1	1/1.000 = 1.000	1	1.000	3	3.000
2	1/1.585 = 0.631	2	1.262	3	1.893
3	1/2.000 = 0.500	0	0.000	1	0.500
4	1/2.322 = 0.431	0	0.000	0	0.000
5	1/2.585 = 0.387	0	0.000	2	0.774

$$DCG_5(A) = 1.000 + 1.262 + 0 + 0 + 0 = 2.262 \tag{2.29}$$

$$DCG_5(B) = 3.000 + 1.893 + 0.500 + 0 + 0.774 = 6.167 \tag{2.30}$$

The discount makes position visible. Both systems have a grade-2 book in the top 5, but A places it at rank 2 (contribution $2 \times 0.631 = 1.262$) while B places its grade-2 book at rank 5 (contribution $2 \times 0.387 = 0.774$). Same grade, same book quality, but A earns 63% more credit from it by showing it earlier.

Discounting with Binary Relevance

DCG works even when all relevance grades are binary (0 or 1). In that case, it reduces to summing the discount factors at positions where a relevant document appears. This isolates the effect of rank position from the effect of graded relevance, which helps clarify what each component contributes.

Example: DCG@10 with binary relevance

Treating every relevant document as grade 1 and every non-relevant as grade 0:

	System A	System B
Relevant in top 10	ranks 1, 2, 6, 7, 9	ranks 1, 2, 3, 5, 6, 8
DCG_{10} (binary)	1.000 + 0.631 + 0.356 + 0.333 + 0.301 = 2.621	1.000 + 0.631 + 0.500 + 0.387 + 0.356 + 0.315 = 3.190

System B scores higher because it places relevant documents more densely near the top (3 in the first 3 ranks vs. A's 2 in the first 2 ranks, then a gap). With graded relevance, B's lead grows further because its top-ranked documents also carry higher grades.

Normalized DCG

DCG values depend on the number of relevant documents that exist and on the magnitude of the grades, so raw DCG scores cannot be compared across different queries. A query with five grade-3 documents in the collection can produce a much higher DCG than one with two grade-1 documents, regardless of system quality.

Normalization divides the actual DCG by the best possible DCG for that query: the score achieved by an ideal ranking that places documents in descending order of their grades.

Key Formula: Normalized DCG

$$nDCG_k = \frac{DCG_k}{IDCG_k} \tag{2.31}$$

where $IDCG_k$ is the DCG of the ideal ranking: all relevant documents sorted by decreasing grade, placed at ranks 1 through k.

$nDCG$ always falls between 0 and 1. A value of 1 means the system produced the best possible ranking for that query at cutoff k. A value of 0 means no relevant document appeared in the first k positions.

Example: nDCG@10 for System A and B

The ideal ranking for the CS-foundations need places the three grade-3 books first, then the four grade-2 books, then grade-1 books:

Rank	Ideal grade	Discount	Contribution
1	3	1.000	3.000
2	3	0.631	1.893
3	3	0.500	1.500
4	2	0.431	0.861
5	2	0.387	0.774
6	2	0.356	0.712
7	2	0.333	0.667
8	1	0.315	0.315
9	1	0.301	0.301
10	1	0.289	0.289

$$IDCG_{10} = 10.313 \tag{2.32}$$

The actual DCG@10 values from above (with graded relevance):

$$nDCG_{10}(A) = \frac{4.599}{10.313} = 0.446 \qquad nDCG_{10}(B) = \frac{7.825}{10.313} = 0.759 \tag{2.33}$$

System B achieves 76% of the ideal ranking quality; System A achieves only 45%.

What nDCG Reveals That AP Does Not

The contrast between AP and nDCG on our running example is instructive:

Metric	System A	System B	Winner
AP (binary, full list)	0.473	0.359	A
nDCG@10 (binary)	0.577	0.702	B
nDCG@10 (graded)	0.446	0.759	B

AP favours System A because AP rewards finding many relevant documents (A finds 12 vs. B's 6), and the denominator $|\text{Rel}| = 15$ heavily penalizes B's 9 misses. The nDCG metrics tell a different story because they evaluate only the top 10 positions: within that window, B places more relevant, higher-grade material near the top.

Switching from binary to graded nDCG widens B's lead further. Under binary evaluation, A's rank-1 book and B's rank-1 book both score 1.0 (both are "relevant"). Under graded evaluation, B's grade-3 book at rank 1 scores 3.0 while A's grade-1 book scores only 1.0. Graded relevance captures what a user experiences: not all useful results are equally useful.

nDCG and AP answer different questions

AP asks: "How good is the complete ranking at placing relevant documents high and finding all of them?" It integrates precision over the full result list and penalizes missed documents. nDCG@k asks: "How much value does the user get from reading the first k results?" It evaluates a fixed window, rewards quality ordering within that window, and does not penalize a system for what it fails to retrieve beyond position k.

The right metric depends on the information need. A patent lawyer conducting an exhaustive prior-art search needs high recall and should evaluate with AP or MAP. A web search user or fact-checker who reads a handful of results cares about nDCG@10 or nDCG@5.

Because the logarithmic discount shrinks toward zero, each additional position contributes less to the sum. Beyond a certain depth, nDCG effectively stops changing: a document at rank 50 contributes only $1/\log_2(51) \approx 0.18$ times its grade, while rank 1 contributes the full grade. For this reason, very large values of k add little discriminative power between systems and are rarely used in practice. Standard cutoffs align with how far users actually read: $k = 10$ for a web search results page (the BEIR and MS MARCO benchmarks report nDCG@10), $k = 20$ for TREC Web Track, and $k = 3$ for voice assistants or mobile interfaces.

Practical Notes

nDCG is the standard metric in web search evaluation (where users rarely look past the first page) and in leaderboard-style benchmarks like MS MARCO and BEIR. The cutoff k is chosen to match the application. The grading scale affects what nDCG rewards. Our linear scale (0, 1, 2, 3) treats the jump from grade 0 to grade 1 as equivalent to the jump from grade 2 to grade 3. An exponential gain variant replaces rel_i with $2^{rel_i} - 1$, which amplifies the difference between high and low grades: a grade-3 document contributes $2^3 - 1 = 7$ while a grade-1 document contributes only $2^1 - 1 = 1$.

Example: Exponential gain variant

Applying the $2^{rel_i} - 1$ gain function to our systems (grade 1 gains 1, grade 2 gains 3, and grade 3 gains 7):

	Linear gain $nDCG_{10}$	Exponential gain $nDCG_{10}$
System A	0.446	0.358
System B	0.759	0.804

System B's lead widens under exponential gain. Its three grade-3 books (gain 7 each) earn far more than A's grade-1 and grade-2 books at the top. System A's score drops because its rank-1 book ("Computer Organization and Design", grade 1) contributes gain $2^1 - 1 = 1$ instead of the linear value 1; the ratio to a grade-3 book jumps from 1:3 to 1:7.

The exponential variant is the default in many evaluation toolkits and benchmark leaderboards. When comparing published nDCG scores, always check whether linear or exponential gain was used; the two are not comparable.

Hands-on: Graded Evaluation

Compute CG, DCG, and nDCG from the library rankings with graded judgments. Swap documents between positions and observe how discounting amplifies or dampens the effect. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Evaluating Text Classifiers

Retrieval systems do not exist in isolation. A search engine for images relies on classifiers that detect faces, identify objects, or assign scene categories before any query arrives. An audio retrieval system uses a classifier to segment speech from music. A filtering stage in a retrieval pipeline decides "does this candidate pass the criteria?": a binary classification decision embedded inside retrieval. These classifiers are components of the retrieval pipeline, and they need their own evaluation. The confusion matrix and its metrics appear every time a system makes a binary or multiclass decision, whether that decision is the final answer or a preprocessing step that feeds into ranking.

The underlying evaluation question is familiar: how often does the system make the right decision? Precision and recall still apply. What changes is the framing. There is no "retrieved set" to measure against a collection; instead, the system assigns exactly one label to each item in a test set, and we compare those predictions against ground-truth labels. Three properties make classification evaluation distinct from retrieval evaluation:

1. **No ranking.** The system outputs a label, not a position. There is no notion of "rank 3 is better than rank 10".
2. **Prevalence matters.** The proportion of positive items in the test set affects how we interpret every metric. A rare disease affects 1% of patients; a common spam folder holds 40% junk.
3. **Error costs are often asymmetric.** A false positive in spam filtering deletes a legitimate email; a false negative lets spam through. These are not equally bad.

The Confusion Matrix

The confusion matrix organizes every prediction the system made into a 2x2 table for binary classification. Each item in the test set falls into exactly one cell, depending on the system's prediction and the true label.

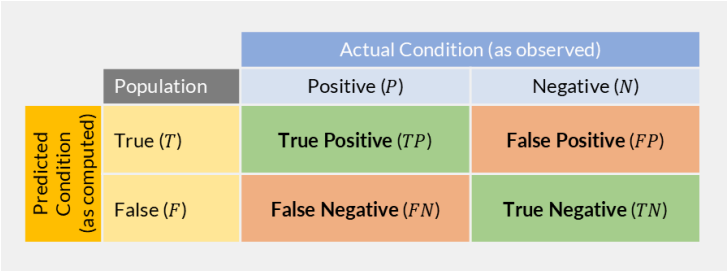


Figure 2.4: The binary confusion matrix. Rows represent predictions, columns represent actual conditions. Green cells are correct predictions; orange cells are errors.

The row sums give the number of items the system labelled positive ($T = TP + FP$) and negative ($F = FN + TN$). The column sums give the actual counts: $P = TP + FN$ positive items and $N = FP + TN$ negative items in the test set. The total population is $P + N$.

From these four counts, we derive the same metrics introduced for retrieval, plus several that become important only in classification:

Key Formula: Classification Metrics

$$\text{Precision (PPV)} = \frac{TP}{TP + FP} \qquad \text{Recall (TPR)} = \frac{TP}{TP + FN} \tag{2.34}$$

$$\text{Specificity (TNR)} = \frac{TN}{TN + FP} \qquad \text{Accuracy} = \frac{TP + TN}{P + N} \tag{2.35}$$

Precision asks: of all items the system called positive, how many truly are? Recall asks: of all truly positive items, how many did the system find? Specificity asks: of all truly negative items, how many did the system correctly leave alone? Accuracy asks: what fraction of all decisions is correct?

Precision and recall are identical to the retrieval definitions from Precision and Recall, just viewed from a different angle: "retrieved" becomes "predicted positive", and "relevant" becomes "actually positive".

Specificity is new; in retrieval we called its complement "fallout" ($FPR = 1 - \text{Specificity}$). Accuracy had no retrieval equivalent because retrieval operates on a ranked list, not a fixed yes/no decision for every item.

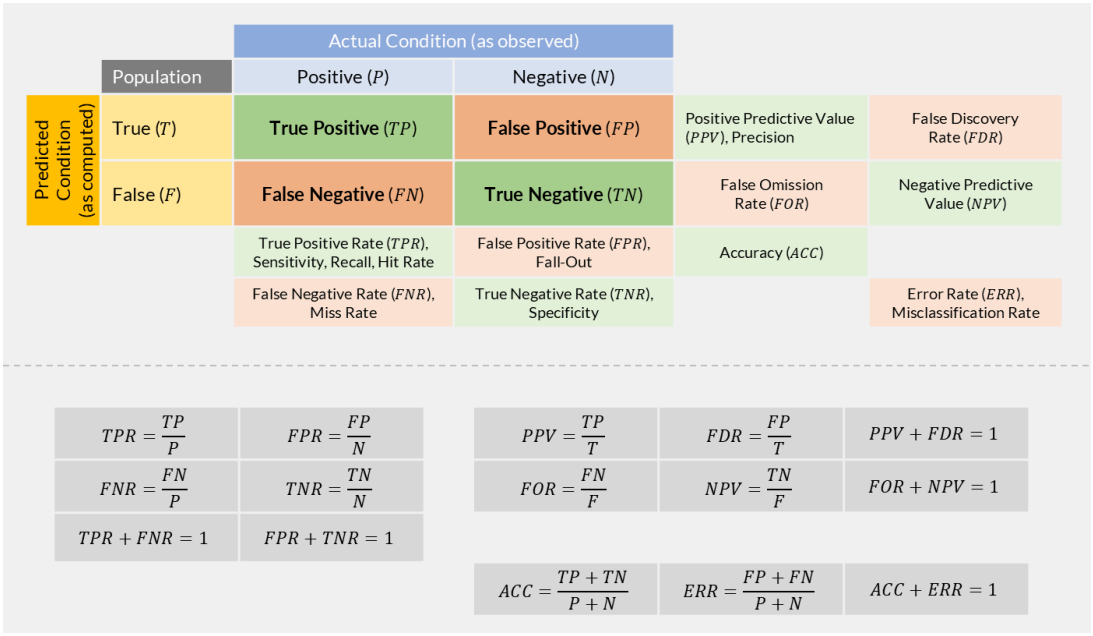


Figure 2.5: The confusion matrix and its derived metrics. Column-normalized rates (TPR, FPR, FNR, TNR), row-normalized predictive values (PPV, NPV), and overall accuracy, with all complementary pairs summing to one.

Example: Spam Filtering

A university email server processes 1000 messages in one day. Of these, 400 are spam ($P = 400$) and 600 are legitimate ($N = 600$). The spam filter makes the following decisions:

	Actual Spam (400)	Actual Legitimate (600)
Predicted Spam	TP = 360	FP = 30
Predicted Legitimate	FN = 40	TN = 570

Example: Spam filter metrics

$$\text{Precision} = \frac{360}{360 + 30} = \frac{360}{390} = 92.3\% \tag{2.36}$$

$$\text{Recall} = \frac{360}{360 + 40} = \frac{360}{400} = 90.0\% \tag{2.37}$$

$$\text{Specificity} = \frac{570}{570 + 30} = \frac{570}{600} = 95.0\% \tag{2.38}$$

$$\text{Accuracy} = \frac{360 + 570}{1000} = 93.0\% \tag{2.39}$$

All four numbers look healthy: the filter catches 90% of spam, and 92% of what it flags is genuinely spam. But consider the 30 false positives: those are 30 legitimate emails moved to the junk folder. If one of them is an acceptance letter for a grant proposal, the cost of that single FP dwarfs the annoyance of 40 spam messages reaching the inbox.

Which error is worse depends on the application

In spam filtering, a false positive (legitimate email deleted) is typically worse than a false negative (spam reaches inbox). The user can ignore spam in the inbox; they cannot read an email they never saw. This asymmetry means that optimizing for high recall (catching all spam) at the cost of precision

(more legitimate emails lost) is the wrong trade-off. A spam filter should favour high precision and high specificity, accepting that some spam slips through.

When Accuracy Misleads

Accuracy counts all correct decisions equally. When the two classes are roughly balanced (as in the spam example with 400 vs. 600), accuracy gives a fair summary. When one class dominates, accuracy becomes misleading.

		Actual Condition (as observed)		
	Population (2030)	Positive ($P = 30$)	Negative ($N = 2000$)	
Predicted Condition (as computed)	True (200)	**True Positive** ($TP = 20$)	**False Positive** ($FP = 180$)	$PPV = \frac{20}{200} = 10\%$
	False (1830)	**False Negative** ($FN = 10$)	**True Negative** ($TN = 1820$)	$NPV = \frac{1820}{1830} = 99\%$
		$TPR = \frac{20}{30} = 67\%$	$TNR = \frac{1820}{2000} = 91\%$	$ACC = \frac{1840}{2030} = 91\%$

Figure 2.6: Confusion matrix for an imbalanced dataset ($P = 30$, $N = 2000$). High accuracy (91%) masks low precision (10%) and moderate recall (67%).

Consider a screening test for a rare disease. In a population of 2030 people, 30 have the disease ($P = 30$) and 2000 do not ($N = 2000$). Figure 2.6 shows the test outcomes: $TP = 20$, $FP = 180$, $FN = 10$, $TN = 1820$.

Example: The accuracy paradox

$$\text{Accuracy} = \frac{20 + 1820}{2030} = \frac{1840}{2030} = 90.6\% \tag{2.40}$$

$$\text{Precision} = \frac{20}{200} = 10\% \qquad \text{Recall} = \frac{20}{30} = 66.7\% \tag{2.41}$$

$$\text{Specificity} = \frac{1820}{2000} = 91\% \qquad \text{NPV} = \frac{1820}{1830} = 99.5\% \tag{2.42}$$

The test is 91% accurate, yet only 10% of positive results are correct. Of every 10 people told they might be sick, 9 are healthy. A trivial baseline that always predicts "healthy" achieves $2000/2030 = 98.5\%$ accuracy, which is better than the actual test.

The problem is prevalence: only $30/2030 = 1.5\%$ of the population is positive. With so few positives, even a small false-positive rate ($FPR = 180/2000 = 9\%$) produces many more false positives than true positives in absolute terms. Accuracy is dominated by the 2000 true negatives and hides the fact that the test is nearly useless for confirming the disease.

When to distrust accuracy

If prevalence is below 10% or above 90%, accuracy alone is uninformative. Report precision, recall, and specificity separately. The F_1-score (or F_β with a chosen β) combines precision and recall without being inflated by true negatives, which makes it a better single-number summary for imbalanced problems.

Asymmetric Error Costs

The spam and screening examples already show that FP and FN carry different costs. A third scenario makes this even starker: biometric face unlock on a smartphone.

The system decides whether the face in front of the camera matches the phone's owner. Two errors are possible:

- **False Positive**: an impostor's face is accepted. The phone unlocks for a stranger. This is a security breach.
- **False Negative**: the owner's face is rejected. The owner must retry or use a PIN. This is a minor inconvenience.

The costs are radically different: a single FP compromises all data on the device, while a FN costs five seconds. The system must therefore achieve an extremely low false-positive rate (FPR), even if this means a noticeable false-negative rate (FNR).

Example: Biometric face unlock

A face-unlock system is tested on 10,000 unlock attempts: 9,000 by the legitimate owner and 1,000 by impostors.

	Owner (9000)	Impostor (1000)
Unlocked	TP = 8820	FP = 2
Rejected	FN = 180	TN = 998

$$\text{FPR} = \frac{2}{1000} = 0.2\% \qquad \text{FNR} = \frac{180}{9000} = 2.0\% \tag{2.43}$$

$$\text{Precision} = \frac{8820}{8822} = 99.98\% \qquad \text{Recall} = \frac{8820}{9000} = 98.0\% \tag{2.44}$$

The system accepts a 2% owner-rejection rate to keep impostor acceptance at 0.2%. This trade-off is deliberate: the security requirement demands $FPR < 0.5\%$ regardless of the cost to FNR.

The choice of which error to minimize is not a mathematical question; it is a design decision that precedes evaluation. The metrics merely reveal whether the system meets the requirement. In the next section, we explore how moving a decision threshold trades FPR against FNR continuously, and how ROC curves visualize this trade-off.

Multiclass Classification

Binary classification assigns one of two labels. Many tasks assign one of K labels: an image classifier distinguishes "indoor", "outdoor", and "portrait"; an AI assistant routes user queries to the correct handler. The confusion matrix generalizes to a $K \times K$ table where rows are predicted classes and columns are actual classes.

Example: Intent routing

An AI assistant classifies each user query into one of four intents: `knowledge_base` (answer from stored documents), `web_search` (look up current information), `calculator` (compute a numerical answer), or `clarify` (ask the user for more detail). Evaluated on 200 queries:

		Actual Class			
	Queries (200)	KB (80)	Web (60)	Calculator (60)	Clarify (20)
Predicted Class	KB (92)	70	8	2	12
	Web (60)	4	48	6	2
	Calculator (36)	2	3	30	1
	Clarify (12)	4	1	2	5

Figure 2.7: Confusion matrix for an intent-routing classifier on 200 queries. Diagonal cells (green) are correct; off-diagonal cells (orange/red) are misrouted queries.

Overall accuracy: $(70 + 48 + 30 + 5)/200 = 76.5\%$. The system routes three out of four queries correctly, but the per-class story is far less uniform.

Per-Class Metrics via One-vs-Rest

To compute precision, recall, and specificity for one class, collapse the $K \times K$ matrix into a binary view: "belongs to class C" vs. "does not belong to class C". Figure 2.8 shows this collapse for two classes from the intent-routing example.

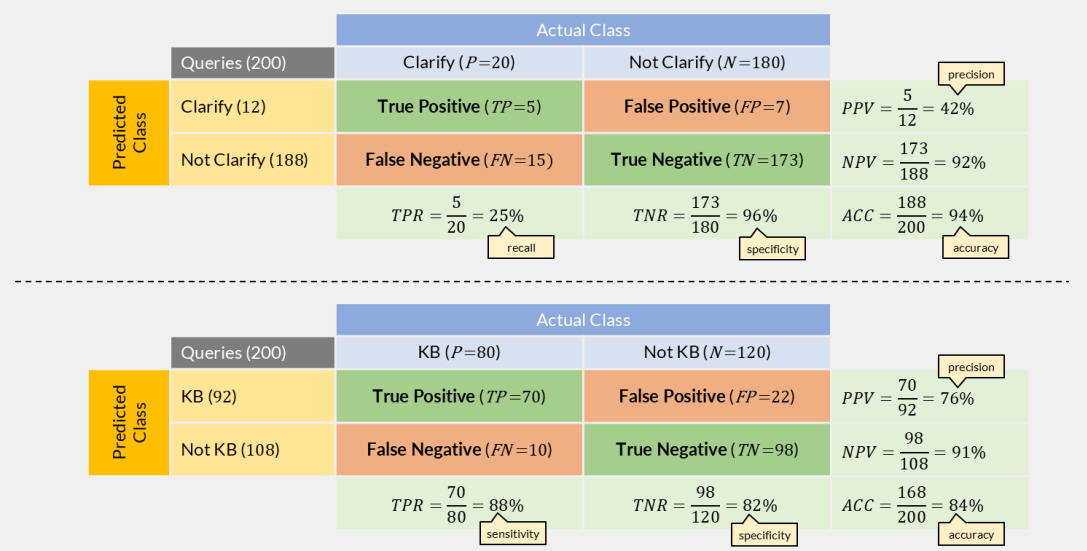


Figure 2.8: One-vs-rest collapse of the intent-routing confusion matrix for the `clarify` class (top) and the `knowledge_base` class (bottom), with derived metrics.

For the `clarify` class (the rare intent with 20 actual queries), the system correctly identifies only one in four queries that actually need clarification. Most of the rest (12 out of 15 missed) get misrouted to `knowledge_base`, where the system attempts to answer a question it does not understand. This is the most dangerous failure mode: the user gets a confident but wrong answer instead of a helpful follow-up question.

For the dominant `knowledge_base` class (80 of 200 queries), the system routes most KB queries correctly (high recall) but also absorbs queries that belong elsewhere (22 false positives), diluting precision. The high recall on the dominant class is what inflates the accuracy of the 4x4 confusion matrix to 76.5%.

Aggregating Across Classes

With K classes, we have K precision values and K recall values. Summarizing them into one number uses the same macro/micro distinction from Precision and Recall:

Macro-averaging computes the metric per class, then takes the arithmetic mean:

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} \text{Precision}_k \tag{2.45}$$

Each class contributes equally, regardless of size. For our intent router:

Class	Precision	Recall
`knowledge_base`	0.761	0.875
`web_search`	0.800	0.800
`calculator`	0.833	0.750
`clarify`	0.417	0.250

$$\text{Precision}_{\text{macro}} = \frac{0.761 + 0.800 + 0.833 + 0.417}{4} = 0.703 \tag{2.46}$$

$$\text{Recall}_{\text{macro}} = \frac{0.875 + 0.800 + 0.750 + 0.250}{4} = 0.669 \tag{2.47}$$

Micro-averaging pools TP, FP, and FN across all classes before computing the ratio:

$$\text{Precision}_{\text{micro}} = \frac{\sum_k TP_k}{\sum_k (TP_k + FP_k)} = \frac{70 + 48 + 30 + 5}{92 + 60 + 36 + 12} = \frac{153}{200} = 0.765 \tag{2.48}$$

$$\text{Recall}_{\text{micro}} = \frac{\sum_k TP_k}{\sum_k (TP_k + FN_k)} = \frac{70 + 48 + 30 + 5}{80 + 60 + 40 + 20} = \frac{153}{200} = 0.765 \tag{2.49}$$

Both denominators equal the total number of queries (200): the precision denominator sums all row totals, the recall denominator sums all column totals, and in a single-label setting these are the same. The numerator is always the sum of the diagonal (correct predictions). So micro-precision, micro-recall, and accuracy all collapse to the same value: diagonal sum divided by total.

This explains why accuracy is the dominant single-number metric in classification: it is the micro-averaged precision and recall. In retrieval, micro-averaging played a secondary role because the macro view (one score per query, then average) better matched evaluation practice. In classification, the micro view is the natural default, and accuracy is its name.

Macro-averaging tells the opposite story. It gives each class equal weight: mean precision drops to 70.3% and mean recall to 66.9%, pulled down by `clarify`'s dismal 25% recall. If every intent matters equally (and it should, because a misrouted clarify query produces a wrong answer), macro-averaging is the right summary.

Accuracy hides per-class failures

Accuracy (= micro-average) is the standard headline metric for classification, and it naturally weights each class by its frequency: being good at the most common task counts more. This is often what we want, since the common case dominates user experience. However, when class sizes differ substantially, accuracy can mask catastrophic failures on rare but important classes. In our example, a 25% recall on `clarify` is invisible in the 76.5% accuracy number. Supplement accuracy with macro-averaged metrics or per-class breakdowns whenever a rare class carries disproportionate cost (a misrouted clarify query produces a wrong answer; a missed fraud case costs millions).

Hands-on: Classification Evaluation

Build confusion matrices for spam filtering and multiclass scenarios, compute per-class metrics, and observe how prevalence affects accuracy. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

All metrics in this section assume a fixed decision: the system outputs a label and we count outcomes. But most classifiers internally produce a continuous score (a probability, a distance, a similarity), and the label emerges only after applying a threshold. Moving that threshold changes the balance between FP and FN. The next section explores this continuous perspective through score distributions, threshold selection, and ROC curves.

Scores, Thresholds, and ROC Curves

The previous section evaluated classifiers after the decision was already made: the system assigned a label, and we counted outcomes. But most classifiers do not jump directly to a label. They produce a continuous score, a confidence value between 0 and 1, and the label emerges only after comparing that score to a threshold. The spam filter assigns each email a spam probability: 0.92 for obvious junk, 0.03 for a message from a colleague, 0.47 for a newsletter that could go either way. At threshold 0.5, the newsletter stays in the inbox; at threshold 0.4, it moves to junk.

Different thresholds produce different confusion matrices, different precision/recall values, and different TPR/FPR trade-offs. A single classifier is not one point in metric space; it is a family of points, one for each threshold. This section explores how to visualize that family and choose a single operating point.

Score Distributions and the Threshold

A binary classifier assigns a score $s(x) \in [0, 1]$ to each item x. Items from the positive class tend to receive higher scores; items from the negative class tend to receive lower scores. If the two score distributions were perfectly separated (all positives above some value, all negatives below), any threshold between them would classify perfectly. In practice, the distributions overlap.

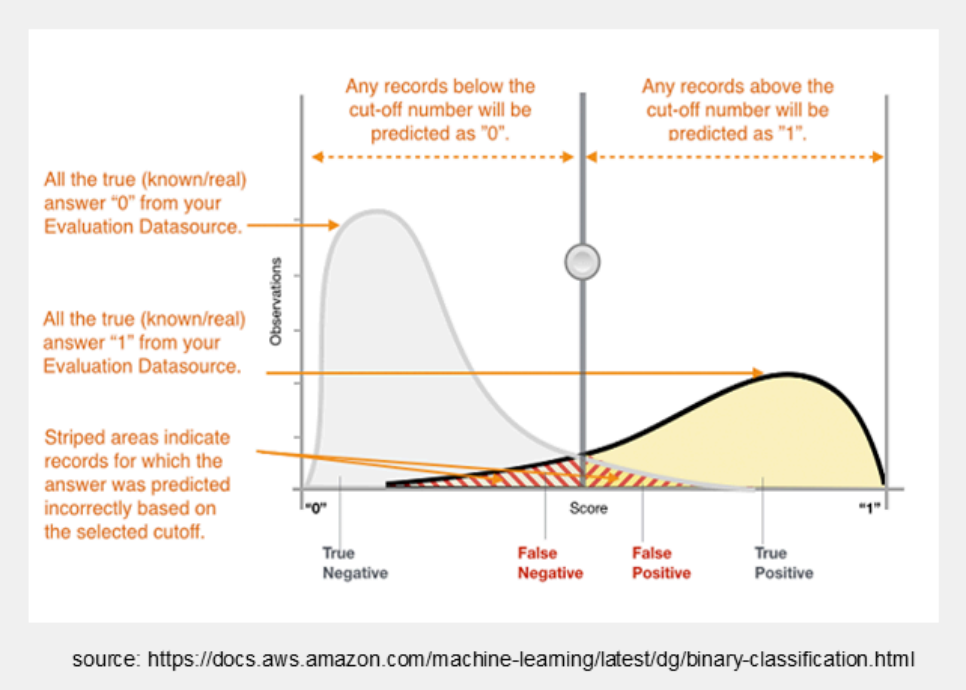


Figure 2.9: Score distributions for the negative class (grey) and positive class (yellow) with a decision threshold. The overlap region produces classification errors regardless of where the threshold is placed.

The threshold T partitions the score axis into two regions: items with $s(x) \geq T$ are predicted positive, items with $s(x) < T$ are predicted negative. Moving the threshold changes the balance between true positives, false positives, true negatives, and false negatives:

- **Shifting T to the left** (lower threshold): more items cross into the "predicted positive" region. TPR increases (fewer false negatives), but FPR also increases (more false positives). The system becomes more sensitive but less specific.
- **Shifting T to the right** (higher threshold): fewer items are predicted positive. FPR decreases (fewer false positives), but TPR also decreases (more false negatives). The system becomes more specific but less sensitive.

This is the fundamental trade-off underlying every threshold-based classifier. No single threshold is universally best; the right choice depends on the error costs of the application.

Formalizing the Threshold as Areas Under the Distributions

Let $f_n(x)$ denote the score distribution of the negative class and $f_p(x)$ the score distribution of the positive class. Each classification rate corresponds to an area under one of these curves, partitioned by the threshold T:

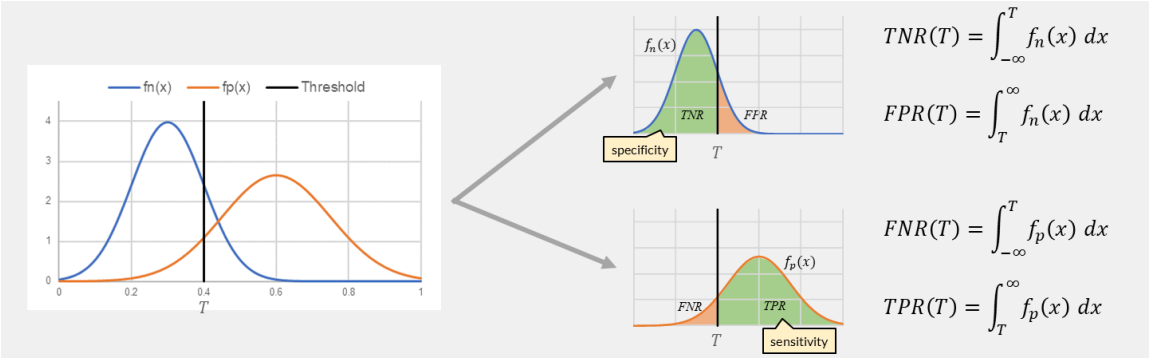


Figure 2.10: Decomposition of the score distributions into the four classification rates. TNR and FPR partition the negative-class curve $f_n(x)$; FNR and TPR partition the positive-class curve $f_p(x)$.

Key Formula: Classification Rates as Integrals

$$TPR(T) = \int_{T}^{\infty} f_p(x)\,dx \qquad FNR(T) = \int_{-\infty}^{T} f_p(x)\,dx \tag{2.50}$$

$$TNR(T) = \int_{-\infty}^{T} f_n(x)\,dx \qquad FPR(T) = \int_{T}^{\infty} f_n(x)\,dx \tag{2.51}$$

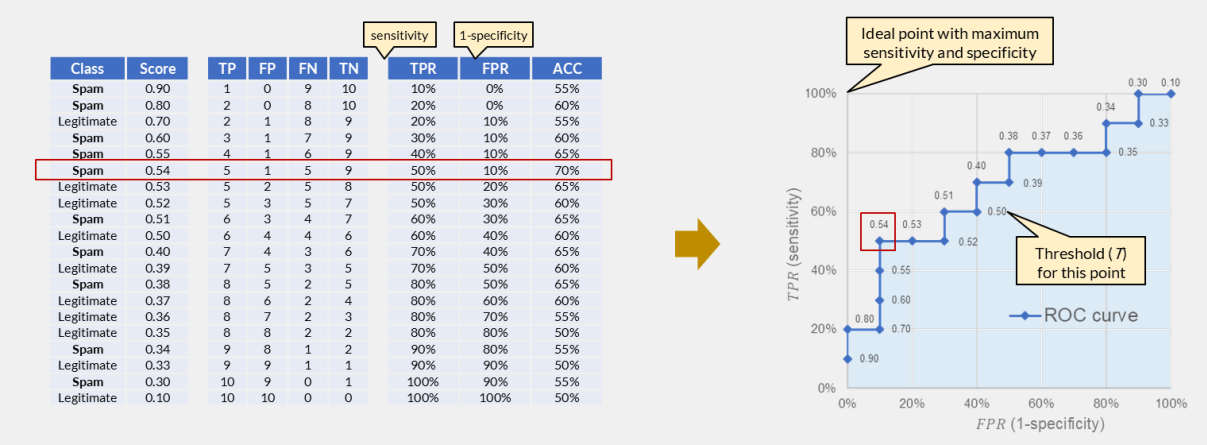
Each pair partitions one distribution: $TPR + FNR = 1$ and $TNR + FPR = 1$. Moving T shifts the boundary between the two areas in each pair.

The integrals make the trade-off precise. Shifting T to the left expands the integration range for TPR (the area under f_p to the right of T grows), but simultaneously expands the integration range for FPR (the area under f_n to the right of T also grows). No threshold eliminates the overlap region; it can only choose how to distribute the overlap errors between false positives and false negatives.

The ROC Curve

Instead of evaluating one threshold, the ROC (Receiver Operating Characteristic) curve evaluates all of them simultaneously. It plots $TPR(T)$ on the vertical axis against $FPR(T)$ on the horizontal axis, sweeping T from high to low. The result is a curve from $(0, 0)$ to $(1, 1)$ that shows the full trade-off landscape.

Figure 2.11 shows 20 emails (10 spam, 10 legitimate) sorted by descending spam score. At each score, we set the threshold there and count cumulative TP, FP, FN, TN. The right panel plots the resulting ROC curve.



Class	Score	TP	FP	FN	TN	TPR	FPR	ACC
Spam	0.90	1	0	9	10	10%	0%	55%
Spam	0.80	2	0	8	10	20%	0%	60%
Legitimate	0.70	2	1	8	9	20%	10%	55%
Spam	0.60	3	1	7	9	30%	10%	60%
Spam	0.55	4	1	6	9	40%	10%	65%
Spam	0.54	5	1	5	9	50%	10%	70%
Legitimate	0.53	5	2	5	8	50%	20%	65%
Legitimate	0.52	5	3	5	7	50%	30%	60%
Spam	0.51	6	3	4	7	60%	30%	65%
Legitimate	0.50	6	4	4	6	60%	40%	60%
Spam	0.40	7	4	3	6	70%	40%	65%
Legitimate	0.39	7	5	3	5	70%	50%	60%
Spam	0.38	8	5	2	5	80%	50%	65%
Legitimate	0.37	8	6	2	4	80%	60%	60%
Legitimate	0.36	8	7	2	3	80%	70%	55%
Legitimate	0.35	8	8	2	2	80%	80%	50%
Spam	0.34	9	8	1	2	90%	80%	55%
Legitimate	0.33	9	9	1	1	90%	90%	50%
Spam	0.30	10	9	0	1	100%	90%	55%
Legitimate	0.10	10	10	0	0	100%	100%	50%

Figure 2.11: ROC curve construction from a ranked list of 20 emails. Each row sets the threshold at that score; the curve traces TPR vs. FPR as the threshold sweeps from high to low. The highlighted point (T = 0.54) has the highest accuracy but is not necessarily the best operating point.

Example: ROC curve construction for a spam filter

The highlighted row at score 0.54 gives $TPR = 50\%$, $FPR = 10\%$, accuracy $= 70\%$. This is the highest-accuracy threshold. But is it the best choice for a spam filter?

- At threshold 0.54: half the spam is caught, and only 10% of legitimate emails are falsely flagged.
- At threshold 0.38: $TPR = 80\%$, $FPR = 50\%$. Most spam is caught, but half the legitimate mail goes to junk.
- At threshold 0.80: $TPR = 20\%$, $FPR = 0\%$. No legitimate email is lost, but 80% of spam reaches the inbox.

For a spam filter, a false positive (legitimate email deleted) is far more costly than a false negative (spam reaching the inbox). The right operating point is not the one that maximizes accuracy but the one that keeps FPR very low while maintaining acceptable TPR. Figure 2.11 shows that no threshold achieves both goals well: the classifier's score separation is simply too weak. This is the point where evaluation tells us to go back and improve the model itself rather than searching for a better threshold on a mediocre curve.

Key Landmarks

Point	Meaning
$(0, 0)$	Threshold so high that nothing is predicted positive (always predict negative)
$(1, 1)$	Threshold so low that everything is predicted positive (always predict positive)
$(0, 1)$	Perfect classifier: all positives detected, no false positives
Diagonal	Random classifier: TPR = FPR at every threshold (no discrimination)

A curve that hugs the upper-left corner represents a strong classifier: it achieves high TPR before FPR rises appreciably. A curve that follows the diagonal adds no information beyond random guessing.

Reading the ROC Space

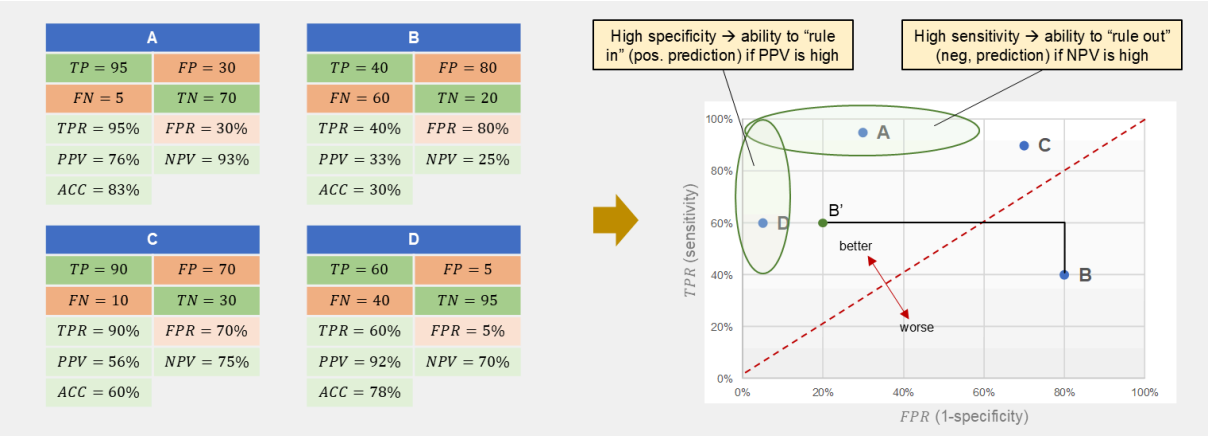


Figure 2.12: Four classifiers (A-D) mapped from their confusion matrices to ROC space. The upper-left region represents high sensitivity with low false-positive rate; the diagonal represents random performance.

Figure 2.12 places four classifiers at different operating points:

- **Classifier A** (TPR = 95%, FPR = 30%): high sensitivity, moderate false-positive rate. If a negative prediction from A is almost certainly correct (high NPV), A can "rule out" the condition: a negative result is trustworthy.
- **Classifier B** (TPR = 40%, FPR = 80%): below the diagonal. This classifier is worse than random. Flipping its predictions gives B' (TPR = 60%, FPR = 20%), which is above the diagonal and usable.
- **Classifier C** (TPR = 90%, FPR = 70%): high sensitivity but very high false-positive rate. It catches most positives but at enormous cost in false alarms.
- **Classifier D** (TPR = 60%, FPR = 5%): high specificity, moderate sensitivity. If a positive prediction from D is almost certainly correct (high PPV), D can "rule in" the condition: a positive result is trustworthy.

The biometric face-unlock from Evaluating Text Classifiers would appear as a point near the left edge of ROC space: $FPR = 0.2\%, TPR = 98\%$. The requirement "FPR below 0.5%" constrains the operating point to a narrow vertical strip on the left side of the plot.

AUC: Area Under the ROC Curve

The ROC curve shows performance across all thresholds. Summarizing it into a single number gives the Area Under the Curve (AUC).

Key Formula: AUC

$$AUC = \int_0^1 TPR(FPR)\, d(FPR) \tag{2.52}$$

Geometrically: the area under the ROC curve. Probabilistically: the probability that a randomly chosen positive item receives a higher score than a randomly chosen negative item. The equivalence holds because constructing the ROC curve is the same as asking, for each negative item (which produces one ΔFPR step to the right), what fraction of positives score higher (the current TPR height). Summing height times width over all negative items gives both the area and the Wilcoxon-Mann-Whitney U-statistic, which is exactly $P(s(x_+) > s(x_-))$ (Hanley and McNeil, 1982).

AUC	Interpretation
1.0	Perfect separation: every positive scores higher than every negative
0.5	Random: scores carry no discriminative information
< 0.5	Worse than random (flip predictions to improve)

Example: AUC interpretation

The AUC can be computed from the step-function ROC curve by summing rectangles: each time a negative item is encountered (FPR steps right by $1/N$), the rectangle's height is the current TPR. For the 20-email spam filter in Figure 2.11, the ROC curve hugs the left edge for the first two spam emails (TPR rises to 20% with FPR still at 0%), then begins stepping right. The resulting AUC is approximately 0.68.

This means: if we pick one random spam email and one random legitimate email, there is a 68% chance the spam email receives the higher score. The imperfect separation arises because several spam and legitimate emails have scores in the 0.30-0.55 range where the distributions overlap heavily.

AUC is useful for comparing classifiers without committing to a threshold: a system with higher AUC has better score separation overall. However, AUC does not say which operating point the system will use in production, and two classifiers with the same AUC can have very different curves (one may be better at low FPR, the other at high TPR).

AUC summarizes but does not prescribe

A classifier with AUC = 0.95 is not necessarily better than one with AUC = 0.90 for a specific application. If the requirement is $FPR < 0.1\%$ (as in biometric unlock), the only relevant part of the curve is the far-left edge. A system with AUC = 0.90 that achieves $TPR = 97\%$ at $FPR = 0.1\%$ is better for that use case than a system with AUC = 0.95 that only reaches $TPR = 85\%$ at the same FPR constraint. Always examine the curve at the operating region that matters.

Choosing an Operating Point

The ROC curve shows what is possible. Selecting one point on it requires a decision criterion from outside the mathematics:

Maximize accuracy. Compute $(TP + TN)/(P + N)$ at each threshold and pick the maximum. This works when classes are balanced and error costs are symmetric. In the spam example from Figure 2.11, threshold 0.54 gives $TP = 5$, $TN = 9$, accuracy $= 14/20 = 70\%$, the highest value in the table. But as we discussed, the spam filter should rather optimize for a low FPR while maintaining acceptable TPR, not simply maximize accuracy.

Constrained optimization. Fix one rate and optimize the other. The face-unlock requirement "$FPR \leq 0.2\%$" defines a vertical line on the ROC plot; the operating point is the highest TPR value at or to the left of that line. This is the standard approach when one error type has a hard cost limit.

Youden's J-statistic. Maximize $J = TPR - FPR$, which is the vertical distance from the diagonal (random classifier). The point with the highest J is geometrically closest to the upper-left corner and balances sensitivity against specificity without committing to a cost model.

Equal error rate (EER). Find the threshold where $FPR = FNR$ (equivalently, $FPR = 1 - TPR$). This is the point where the ROC curve crosses the anti-diagonal from $(0, 1)$ to $(1, 0)$. EER is commonly reported in biometric and speaker-verification systems as a single-number summary that does not require choosing which error is worse.

In all cases, threshold selection should be performed on a validation set separate from the test set used to report final metrics. Optimizing the threshold on the test set inflates reported performance.

Hands-on: Thresholds and ROC

Sweep the decision threshold across the spam-filter scores and watch the confusion matrix, TPR, FPR, and ROC curve update in real time. Compute AUC and find the threshold that maximizes accuracy vs. the one that satisfies a FPR constraint. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

When Evaluation Misleads

Two results arrive on the same afternoon. A retrieval group reports that their new ranker raises nDCG@10 on a public benchmark from 0.41 to 0.43. A colleague reports that their spam filter reaches 99.2% accuracy on a held-out test set. Both numbers were computed correctly with the measures developed in the previous sections. Both might still be worthless.

Now that the measures are in hand, we can look at what they do not tell us. Every number in this chapter rests on a chain of decisions: which documents were collected, which needs were written down, who judged them, how the labels were split, and which measure was reported. A weakness anywhere in that chain reaches the final score, and the score itself never reveals it. Reading evaluation results critically means knowing where the chain tends to break.

Incomplete Ground Truth

Recall the pooling arrangement from Designing a Retrieval Benchmark: at realistic scale, only the documents that participating systems actually retrieved get judged, and everything unjudged counts as not relevant. That convention is what makes large benchmarks affordable, and it is also the first place scores go wrong.

In the **dense case** (TREC-style pooling), the benchmark assessed all documents that participating systems submitted in a given year. The judgments are thorough for those systems. But when a new algorithm is evaluated against the same reused collection, it may retrieve relevant documents that no original participant found. Those documents were never assessed and count as non-relevant. The new system is penalized for being original: it found material the pool missed, and the benchmark cannot reward it. A careless evaluator might even conclude the new system is worse, when in fact it discovered answers nobody had seen before.

In the **sparse case** (MS MARCO-style benchmarks with millions of queries), each query has only one or a few assessed relevant documents. Figure 2.13 shows this situation. Any document that equally qualifies as a correct answer but was never selected for assessment penalizes rather than rewards the system that retrieves it. The bias is systematic: retrieval systems trained on such benchmarks tend to replicate what was selected for assessment rather than what is relevant in a broader sense.

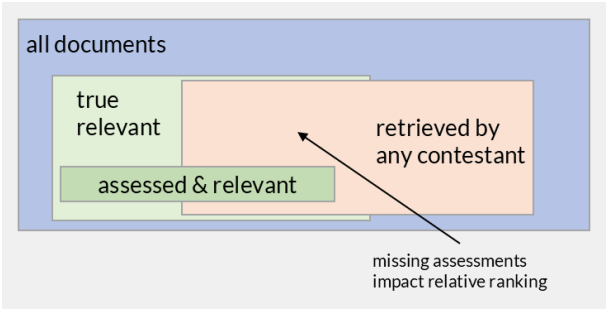


Figure 2.13: Sparse assessment: only a fraction of genuinely relevant documents were ever judged. Systems retrieving unjudged relevant material are penalized.

Recall suffers most, and in a direction that depends on who ran the experiment. Precision at rank 10 needs only the top ten documents judged, which pooling usually delivers. Recall needs the total amount of relevant material in the collection, and that is exactly what incomplete judging never establishes. For the systems that built the pool, recall comes out too high, because the divisor counts only the relevant documents somebody found. For a system evaluated later against the reused collection, the error can run the other way, since its own relevant results may be unjudged and scored as misses.

Unjudged is not the same as not relevant

Every metric in this chapter treats an unjudged document as non-relevant, because the arithmetic needs a label and there is none. That is a convention forced by cost, not a finding about the document. When comparing two systems on a sparsely judged benchmark, a lower score can mean “retrieved different documents” rather than “retrieved worse documents”.

Noisy Ground Truth

Even judged labels are opinions. Assessors work from written guidelines and still disagree on borderline cases, so a benchmark encodes one panel's reading of each need. This is tolerable for comparison, since every system is scored against the same labels, but it puts a floor under how small a difference we can meaningfully interpret. A gap of one point in a measure whose ground truth two qualified assessors would not fully reproduce is not a result.

Classification test sets have the same problem in a different shape. Their labels are usually complete, so pooling bias does not apply, but completeness is not correctness. Whoever labelled the messages had to decide whether an aggressive marketing newsletter counts as spam, and different labellers draw that line differently. A classifier that learned one labeller's line will look better than one that learned another's, independently of which is more useful.

Rare classes make this worse because small counts are unstable. In an intent router with four routes, a test set may contain hundreds of knowledge-base queries and a dozen calculator queries. Per-class recall for the calculator route then moves by eight percentage points when a single example changes label, and a macro-averaged score built from such classes inherits that noise. Before trusting a per-class number, check how many test items it was computed from.

Optimizing Against the Test

A benchmark that stays fixed for years stops being a test and becomes a target. Recall how a TREC-style evaluation runs: participants receive the collection and the topics, run their systems on their own machines, and submit ranked lists that the organizers judge afterwards. Holding the topics is what makes the risk possible. A research group can read the topics, notice that several hinge on ambiguous words, add a hand-built list of expansion terms, adjust the stop word list, add exceptions to the stemmer, and keep every change that raises the score. Each step on its own looks like sound engineering. The result is a system that has absorbed the properties of this topic set and this collection: which words happen to be ambiguous here, and which documents happen to be relevant. The score rises and the method does not transfer. No dishonesty is required, because the selection pressure alone is enough.

Issuing a new topic set for each cycle limits the damage while a benchmark is live, and TREC does this. The cost is comparability, since results measured on this year's topics cannot be placed directly against last year's. The concentrated risk appears after a cycle closes, when the collection, its topics, and its judgments become a reusable test collection that papers measure against for years. Armstrong, Moffat, Webber, and Zobel examined a decade of published ad-hoc retrieval results obtained that way in **Improvements That Don't Add Up**. Dozens of papers reported gains and many claimed statistical significance, yet the improvements did not accumulate into visible progress, and the baselines were often weaker than the median system from the original TREC cycle. Reported improvement and real improvement had come apart.

Classification has a sharper and more mechanical version of the same failure. A decision threshold tuned on the test set produces a score that cannot be trusted, because the threshold was chosen with knowledge of the answers. This is why the validation split exists, and why the threshold selection in Scores, Thresholds, and ROC Curves must happen on validation data. The extreme case is contamination: if the test items and their labels were part of a model's training data, the model can score well by recalling an answer rather than by solving the task. Large language models make this problem acute, because they are trained on web-scale corpora that include publicly available exam questions. When GPT-4 is evaluated on the SAT or the MMLU benchmark, the questions and their correct answers were likely present somewhere in the training data. Researchers have shown that GPT-4 can guess missing answer options in MMLU questions with a 57% exact-match rate, far above chance, which strongly suggests memorization rather than reasoning (Investigating Data Contamination in Modern Benchmarks for Large Language Models, 2023). A high score on a contaminated benchmark tells us the model saw the answers before, not that it can generalize to unseen problems.

Withholding the labels is a partial defence. MS MARCO keeps the labels of its evaluation set private and scores submissions centrally, so participants cannot tune against them directly. The leaderboard's own designers describe the limit in **Fostering Coopetition While Plugging Leaks**: every submission returns

a score, and every score leaks a little information about the hidden set. Enough attempts turn a blind test into a slowly revealed one, which is why the official metric carries a deliberate artifact meant to frustrate reverse-engineering.

A genuinely blind evaluation reverses the direction in which data travels. Instead of sending the topics to the participant, the participant sends the system to the evaluator: code or a container executed against topics it has never seen, with judgments made only after all results are collected. This is known as evaluation as a service, or software submission. **TIRA** implements it by keeping the test data and ground truth out of public reach and running each submitted system in a sandbox that prevents data leaks. The protocol costs the organizers considerably more effort, which is why it remains the exception. One caveat survives even then: a blind topic set shows that a system never saw these needs, but for a model pretrained on web text it cannot show that the documents themselves were absent from training.

A benchmark score is not a claim about generalization

A score describes how a system performs on one collection, one topic set, and one panel's judgments, usually under a protocol that let the system's authors read the topics first. Whether the method transfers elsewhere is a separate claim needing separate evidence: a different collection, a topic set the authors never inspected, or a blind evaluation.

The Test Set Is Not Production

Benchmarks are frozen so that results stay reproducible, and freezing is what makes them drift away from the world they were drawn from. Vocabulary shifts, topics appear that the collection predates, document formats change. The stability that makes a benchmark useful for comparison steadily reduces how much it says about current traffic.

For classification the mismatch is measurable, and prevalence is the clearest case. Suppose a spam filter detects 90% of spam and correctly passes 98% of legitimate mail. On a balanced test set of 500 spam and 500 legitimate messages, it produces 450 true positives and 10 false positives, so precision is $450/460 \approx 97.8\%$. Deploy the same filter, unchanged, on a mailbox where only 5% of messages are spam. Out of 500 spam and 9,500 legitimate messages it still catches 450 and still misclassifies 2% of legitimate mail, but 2% of 9,500 is 190 false positives, so precision falls to $450/640 \approx 70.3\%$.

Nothing about the classifier changed. Recall and specificity are identical in both settings. Only the class balance moved, and precision, the measure the user actually experiences, dropped by 27 percentage points. A test set whose prevalence does not match deployment therefore reports a precision that deployment will not reproduce.

One Number Is Not a Result

A single score on a single collection is weak evidence, for a reason that has nothing to do with retrieval. Per-need scores vary enormously: the same system can reach average precision above 0.8 on one need and near zero on another. An average over 50 needs conceals that spread, so two systems differing by a point in the mean may be indistinguishable once the variation between needs is taken into account. Reporting the spread, and testing whether an observed difference exceeds it, matters as much as the measure itself.

The problem compounds across a community. When hundreds of groups tune against one public benchmark, the leading entry is partly the winner of a large search over method variants, and some of its margin is the luck of that search rather than a property of the method. This is the same multiple-comparisons effect that makes a single significance test misleading when many hypotheses were tried, and it is one mechanism behind the missing accumulation that Armstrong and colleagues documented.

Effectiveness Is Not the Only Axis

Finally, relevance is one dimension of quality among several. A production system must also answer quickly, sustain many concurrent requests, and stay affordable, and these goals pull against each other. Later chapters on vector search make the trade-off explicit: approximate nearest-neighbour methods deliberately

accept slightly worse results in exchange for large gains in speed and cost, and that exchange is often the right decision. A benchmark reporting only effectiveness cannot express it, which is why operational targets such as latency, throughput, and cost per query belong in the performance goals next to the retrieval measures.

None of this makes evaluation futile. It makes evaluation a claim with a scope. The following questions establish that scope for any reported score:

- How complete are the judgments, and could a better system have been penalized for retrieving unjudged documents?
- Who produced the labels, and how much would a second panel agree?
- Did the authors see the test data before reporting, and was any threshold or hyperparameter chosen using it?
- How closely do the collection and the class balance match the setting where the system will run?
- How large is the difference relative to the variation across needs, and how many alternatives were tried before this one was reported?
- What does the score cost in latency and compute?

A result that survives these questions is worth acting on.

Summary

Metric Lookup Table

Metric	Formula	What it rewards	When to use
Precision	$\frac{TP}{TP+FP}$	Clean results (few false positives)	Any binary decision; especially when FP is costly
Recall	$\frac{TP}{TP+FN}$	Completeness (few misses)	Exhaustive search; patent, legal, medical
F_β	$\frac{(\beta^2+1)\cdot P\cdot R}{\beta^2\cdot P+R}$	Balances P and R with tuneable weight	Single-number summary; $\beta=1$ default
$P@k$	Precision in top k	Clean top-of-list	Web search, short result pages
MRR	Mean of $1/\text{rank}_{\text{first relevant}}$	First relevant result high	QA, known-item search
R-Precision	$P@R$	Balance at a natural cutoff	Self-adjusting single-point summary
AP	Mean of P_k at relevant positions, divided by $\lvert \text{Rel}\rvert$	Both high precision and high recall, position-weighted	Full ranking evaluation per query
MAP	Mean of per-query AP	System-level ranking quality	Benchmark comparison (TREC standard)
$nDCG@k$	$DCG_k/IDCG_k$	High-grade documents placed early	Web search, graded relevance, user experience
Accuracy	$\frac{TP+TN}{P+N}$	Overall correct decisions	Balanced classification; equals micro-P/R
Specificity (TNR)	$\frac{TN}{TN+FP}$	Few false alarms	Medical tests, biometric security
AUC	Area under ROC curve	Score separation across all thresholds	Comparing classifiers without committing to a threshold

Key Takeaways

1. Evaluation requires a benchmark: a collection, information needs, relevance judgments, and a performance goal. Without any component, a number is not interpretable.
2. Precision and recall are in tension. Improving one typically costs the other, and the F_β-measure makes the trade-off explicit.
3. Ranked evaluation adds position: AP and nDCG reward systems that place relevant (or high-grade) material where users will see it.
4. Graded relevance distinguishes "somewhat useful" from "essential". nDCG captures what binary metrics cannot: the quality of the relevant documents, not only their presence.
5. Classification evaluation reuses precision and recall but adds accuracy, specificity, and the confusion matrix. Accuracy equals micro-precision equals micro-recall in single-label settings.
6. Prevalence distorts accuracy: when one class dominates, a trivial baseline can score higher than a real classifier. Always check class balance before trusting accuracy.
7. A continuous score becomes a binary decision only after choosing a threshold. The ROC curve shows all possible trade-offs; AUC summarizes overall score separation.
8. No benchmark score is a claim about generalization. Incomplete judgments, data contamination, prevalence mismatch, and overfitting to static test sets all limit what a number can mean.

Key Formulas

$$P = \frac{TP}{TP + FP} \qquad R = \frac{TP}{TP + FN} \tag{2.53}$$

Precision and recall: the fundamental pair from which all other metrics derive.

$$AP = \frac{1}{|\text{ Rel}|} \sum_{k:d_k \in \text{ Rel}} P_k \tag{2.54}$$

Average precision: area under the precision-recall curve, the standard per-query measure.

$$nDCG_k = \frac{\sum_{i=1}^{k} \frac{rel_i}{\log_2(i+1)}}{\text{IDCG}_k} \tag{2.55}$$

Normalized discounted cumulative gain: the standard for graded relevance evaluation.

$$AUC = P(s(x_+) > s(x_-)) \tag{2.56}$$

Area under the ROC curve: the probability that a random positive scores higher than a random negative.

Exam focus

- The difference between precision and recall, and why both are always reported together
- How AP integrates both precision and recall in one number, and why missed documents hurt AP
- Why nDCG and AP can disagree on which system is better (top-k user experience vs. full-list completeness)
- Why accuracy misleads under class imbalance, and how prevalence affects precision
- How moving a decision threshold trades TPR against FPR, and what the ROC curve visualizes
- The difference between macro-averaging (each query/class counts equally) and micro-averaging (each item counts equally)

Self-Check Questions

1. (Understand) A system achieves 95% accuracy on a dataset where 95% of items belong to class A. Is this system useful? What additional metric would reveal the problem?
2. (Analyze) System X has MAP = 0.60 and System Y has MAP = 0.55, both measured on 50 queries. Under what circumstances might Y actually be the better system?
3. (Evaluate) A face-unlock system reports AUC = 0.99 and a spam filter reports AUC = 0.99. Are they equally good? What additional information do you need to judge each for its intended use?
4. (Apply) Given a ranked list of 10 documents where relevant documents appear at positions 1, 4, and 7 (out of 8 relevant in the collection), compute $P@5$, AP, and explain why AP is less than the average of the three precision values.
5. (Analyze) Explain why macro-averaged recall across 4 information needs can disagree with micro-averaged recall, and give a concrete scenario where the disagreement is large.

Test Your Knowledge

Take the Chapter 2 Quiz ->

Further Reading

- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to Information Retrieval**, Chapter 8: Evaluation in Information Retrieval. *Cambridge University Press*. Free online version. The standard textbook treatment of precision, recall, MAP, and benchmark design.
- Järvelin, K., & Kekäläinen, J. (2002). **Cumulated Gain-Based Evaluation of IR Techniques**. *ACM Transactions on Information Systems*, 20(4), 422-446. Free PDF. Introduced DCG and nDCG for graded relevance evaluation.
- Hanley, J. A., & McNeil, B. J. (1982). **The Meaning and Use of the Area under a Receiver Operating Characteristic (ROC) Curve**. *Radiology*, 143(1), 29-36. Established the probabilistic interpretation of AUC and its statistical properties.

- Fawcett, T. (2006). **An Introduction to ROC Analysis**. *Pattern Recognition Letters*, 27(8), 861-874. Free PDF. The standard tutorial on ROC curves, AUC, and threshold selection for binary classifiers; covers construction, interpretation, and common pitfalls.
- Armstrong, T. G., Moffat, A., Webber, W., & Zobel, J. (2009). **Improvements That Don't Add Up: Ad-Hoc Retrieval Results Since 1998**. *Proceedings of CIKM 2009*, 601-610. Author copy. Documented how benchmark overfitting prevents genuine progress from accumulating.
- Oren, Y., Bandel, E., Bhatt, G., Shmidman, A., & Sharvit, Y. (2023). **Investigating Data Contamination in Modern Benchmarks for Large Language Models**. arXiv:2311.09783. Demonstrated that LLMs can recall benchmark answers from training data, with GPT-4 guessing missing MMLU options at 57% exact-match rate.

Benchmarks and Evaluation Campaigns

- TREC (Text REtrieval Conference). Annual evaluation campaigns run by NIST since 1992. The source of pooled test collections, shared topics, and the MAP-based evaluation methodology used throughout this chapter.
- MS MARCO. Large-scale passage and document retrieval benchmark built from real Bing queries. Standard proving ground for neural retrieval models; reports MRR@10 and nDCG@10.
- BEIR. Zero-shot retrieval benchmark spanning 18 heterogeneous datasets. Tests whether a retrieval model generalizes beyond its training domain; reports nDCG@10.